

Contents

1	Blok 1: Systemy rozproszone — podstawy	1
2	Blok 2: Programowanie równoległe, RPC, systemy plików	3
3	Zarządzanie zasobami: PBS, Slurm, Kubernetes	6
4	Blok 3: Gridy obliczeniowe (HTCondor, DIRAC)	8
5	Blok 4: Przetwarzanie w chmurze	10
6	Blok 5: IoT, Edge i Fog Computing	12
7	Blok 6: Big Data (MapReduce, Hadoop, HDFS)	13

1.1 Blok 1: Systemy rozproszone — podstawy

(Rozwój obliczeń rozproszonych) wczoraj i dziś
Moc obliczeniowa = towar deficytowy. Etapy:

- lata 80: superkomputery MPP (Massive Parallel Processors)
- architektura NUMA (Non-Uniform Memory Access) — jednolita przestrzeń adresowa, niejednorodny czas dostępu
- czynniki ekonomiczne ⇒ budowa z tanich PC: pierwsze klastrów OS
- potem gridy obliczeniowe (geograficznie rozproszone)

Znaczenie: duże zadania (badania naukowe, AI, LLM: ChatGPT, DeepSeek, Gemini) — gdy jedna maszyna nie wystarcza.

(Kamienie milowe)

- ARPANET (1969) — 1. sieć z przełączaniem pakietów = grid
- MOSIX (1977, idea SSI = Single System Image, klastr jak jeden komputer)
- Beowulf — klastry PC w domenie lokalnej
- Globus (następca I-WAY), Legion, NetSolve (RPC)
- PL: Clusterix, UNICORE, PL-Grid
- GGF (Global Grid Forum, 2000) — standaryzacja
- po 2000: AI, wirtualizacja, konteneryzacja, chmura, fog

(Klastry vs Gridy) cechy i klasyfikacja
Klastr: system wielokomputerowy, maszyny powiązane fizycznie i logicznie; każda ma własną kopię OS; tworzą jeden wirtualny komputer. Najczęściej:

- homogeniczne maszyny
- dedykowana sieć, domena lokalna
- wyodrębniony komputer dostępowy

Grid: heterogeniczny system geograficzny, sieć globalna, specjalne middleware (uprawnieni użytkownicy); budowany na bazie klastrów.
OS: UNIX / Linux. SSI = klastr widziany jako 1 komputer.

(Zalety przetwarzania równoległego)

- przyspieszenie obliczeń
- większa dokładność (więcej punktów przy tym samym czasie)
- zadania o dużych zasobach (pamięć)
- większa niezawodność (awaria 1 maszyny ≠ przerwanie)
- dynamiczna konfiguracja (dołączanie/odłączanie maszyn)

(Równoległe vs współbieżne vs rozproszone) różnice

- równoległe — naprawdę jednocześnie, wiele rdzeni/procesorów (podklasa współbieżnych)
- współbieżne — procesy „w trakcie” w tym samym czasie; może być 1 rdzeń z podziałem czasu
- rozproszone — brak jednolitej przestrzeni adresowej (pamięć rozproszona, nie wspólna)

„Procesor” = 1 rdzeń działający sekwencyjnie (pomijamy AVX/hyper-threading).

1.2 Przyspieszenie i prawo Amdahla

(Współczynnik przyspieszenia) speedup

$$S(n, p) = \frac{T(n, 1)}{T(n, p)} \leq p$$
$$T(n, p) = \beta(n) T(n, 1) + \frac{(1 - \beta(n)) T(n, 1)}{p}$$

n — wielkość zadania, p — liczba procesorów. $T(n, 1)$ — czas sekwencyjny (1 rdzeń), $T(n, p)$ — czas na p procesorach. $\beta(n)$ — udział części sekwencyjnej; $1 - \beta(n)$ — część równoległa (zrównoleglona idealnie). Pomija synchronizację i komunikację.

(Prawo Amdahla) ograniczenie

$$S(n, p) = \frac{1}{\beta(n) + \frac{1 - \beta(n)}{p}}$$
$$S_{max} = \lim_{p \rightarrow \infty} S(n, p) = \frac{1}{\beta(n)}$$

Ograniczenie: nawet przy $p \rightarrow \infty$ przyspieszenie $\leq 1/\beta(n)$ (odwrotność udziału sekwencyjnego). Np. $\beta = 10\% \Rightarrow \max 10\times$.

Przyspieszenie idealne $S \rightarrow p$ tylko gdy $\beta(n) = 0$. Często $\beta(n) \xrightarrow{n \rightarrow \infty} 0$ (gdy część seq. stała $\beta_S T(1, 1)$, a równoległa rośnie z n). Można też osłabić obliczeniami asynchronicznymi (brak sztywnego podziału).

(Amdahl — zastosowanie) policz S lub p

$$S = \frac{1}{(1 - P) + \frac{P}{N}} \quad (P = \text{cz. równoległa})$$
$$N = \frac{P}{\frac{1}{S} - (1 - P)} \quad (\text{odwrócone})$$

P — frakcja równoległa ($P = 1 - \beta$), N — liczba procesorów.
Policz S : podstaw P, N . Np. $P = 0,8, N = 4: S = 1/(0,2 + 0,2) = 2,5$.
Policz N dla danego S : odwróć wzór. Np. $P = 0,8, S = 4: N = 0,8/(0,25 - 0,2) = 16$.
Warunek wykonalności: $S < 1/(1 - P)$.

1.3 Bezpieczeństwo

(Filary bezpieczeństwa) rozproszone
System niebezpieczny ⇒ niezawodny. Chronimy poufność i integralność.

- uwierzytelnianie (authentication) — weryfikacja deklarowanej tożsamości
- autoryzacja (authorization) — czy podmiot ma prawa dostępu
- zaufanie (trust) — w jakim stopniu ufać uwierzytelnionemu; autoryzacja ogranicza szkody (np. limit przelewów)

Podstawa: kryptografia.

(Kryptografia: sym. vs asym.) klucze
 $K(\text{data})$ — klucz K operuje na danych. E_K — szyfrowanie, D_K — deszyfrowanie.

- symetryczna: jeden klucz, $D_K(E_K(\text{data})) = \text{data}$, $D_K = E_K$; klucz musi być tajny u obu stron
- asymetryczna (klucz publiczny): para PK (publiczny, dla każdego) + SK (prywatny/tajny)

Poufność: Alicja → Bob: $PK_B(\text{data})$, odczyta tylko $SK_B(PK_B(\text{data}))$.

(Funkcje skrótu i podpisy cyfrowe) hash
Funkcja skrótu H : $H(\text{data})$ — ciąg stałej długości.

- każda zmiana danych ⇒ inny skrót
- z h obliczeniowo niemożliwe odtworzenie danych (jednokierunkowa)

Podpis cyfrowy: Alicja liczy $sig = SK_A(H(\text{data}))$, wysła $\text{data} + sig$. Bob: $PK_A(sig)$ i sprawdza, czy $= H(\text{data})$. Dowodzi tożsamości i integralności.

(Kryptografia w praktyce)

- bezpieczny kanał — wzajemne uwierzytelnianie + poufność (np. HTTPS)
- blockchain — łańcuch bloków łączony skrótami

(Delegowanie uprawnień, tokeny / proxy) Neuman, Kerberos
Problem: klient poczty ma działać „w imieniu” Alicji. **Delegowanie** = przekazanie praw z procesu na proces (lokalnie, taniej, różne maszyny / domeny).
Rozwiązanie (Neuman, Kerberos): **pełnomocnik (proxy)** przez **token** — pozwala działać z tymi samymi lub **ograniczonymi** prawami.
Struktura tokenu (rys. 2.1):

- R — prawa dostępu (access rights)
- PK_{proxy} — część publiczna sekretu
- $sig(A, \{R, PK_{proxy}\})$ — podpis twórcy A (ochrona przed modyfikacją)
- SK_{proxy} — część prywatna (chroniona!)

OAuth 2.1 — protokół delegowania (Amazon, Google, FB, MS, Twitter).

(OAuth 2.1 — role i tokeny) **Role:** właściciel zasobu (Alicja), klient (aplikacja), serwer zasobów, **serwer autoryzacji** (AC).
Przepływ: Klient ma cid ; $send[cid, R, H(S)] \rightarrow$ kod autoryzacyjny **AC**; $send[cid, AC, S] \rightarrow$ **token dostępu AT**.
2 typy tokenów: (1) identyfikator (serwer zasobów pyta AC za każdym razem), (2) podpisany **certyfikat** z prawami + czasem wygaśnięcia.

1.4 Architektury systemów rozproszonych

(Logiczna vs fizyczna) org. oprogramowania

- **architektura oprogramowania (logiczna)** — jak komponenty są zorganizowane i oddziałują (styl architektoniczny)
- **architektura systemu (fizyczna)** — rozmieszczenie instancji komponentów na **rzeczywistych maszynach**

Komponent — modułowa jednostka z dobrze zdefiniowanymi interfejsami, wymiennalna.

Middleware — warstwa pośrednicząca; cel: **przezroczystość rozproszenia**. Systemy: scentralizowane (klient-serwer), zdecentralizowane (P2P), **hybrydowe**.

(Style architektoniczne) 3 główne

- **warstwowe** (layered)
- **zorientowane na usługi** (SOA, obiekty, mikrousługi, REST)
- **publikuj-subskrybuj** (publish-subscribe)

Realne systemy = **mieszanica** stylów; podział na warstwy jest niemal uniwersalny.

(Styl warstwowy) layered
Komponent w warstwie L_i wywołuje warstwę **niższą** L_j ($j < i$) i oczekuje odpowiedzi. Warianty:

- (a) czysty: tylko wywołania w dół (request/response)
- (b) mieszany: pomijanie warstw
- (c) **upcall**: niższa warstwa wywołuje wyższą przez **uchwyt** (handle) — np. OS sygnalizuje zdarzenie

Zastosowanie: stosy protokołów (np. TCP). Każda warstwa = **interfejs** (co) + **protokół** (zasady wymiany).

(Warstwy aplikacji (3-tier))

- **interfejs aplikacji** (UI)
- **przetwarzanie** (logika, np. ranking, generator HTML)
- **dane** (baza / system plików)

Przykład: wyszukiwarka domów (UI $\rightarrow$ query / ranking $\rightarrow$ DB).

(Zorientowane na usługi (SOA)) obiekty, mikrousługi
Łuźna org. = zbiór odrębnych, niezależnych jednostek; każda **hermetyzuje usługę** jako osobny proces / wątek.

- **styl obiektowy:** obiekt = komponent, łączone zdalnym wywołaniem metody (RPC); **obiekt zdalny:** interfejs na 1 maszynie (serwer **proxy** = stub klienta), implementacja na innej (**szkielet** / skeleton serwera). Stan **nie** jest rozproszony.
- **SOA** (Service-Oriented Architecture) — kompozycja usług, gruboziarnista

(Mikrousługi i REST) microservices

Mikrousługi: małe, niezależne procesy sieciowe (inspiracja Unix); klucz = **modularyzacja**; pytanie **orkiestracji**: gdzie umieścić usługę.

REST (Representational State Transfer) — 4 cechy RESTful:

1. zasoby z jednym schematem nazewnictwa
2. ten sam interfejs: ≤ 4 operacje
3. wiadomości **samoopisowe**
4. **bezstanowe** (stateless) — usługa zapomina o wołającym

Operacje: POST (utwórz), PUT (modyfikuj/nowy stan), GET (pobierz), DELETE (usuń). Zaleta: prostota.

1.5 Publikuj-subskrybuj i Linda

(Koordynacja procesów) sprzężenie
Separacja **przetwarzania** i **koordynacji**. Wg sprzężenia czasowego/referencyjnego:

- **bezpośrednia** — sprzężone czasowo i referencyjnie (telefon)
- **skrzynki pocztowe** — oddzielone czasowo, sprzężone referencyjnie
- **oparta na zdarzeniach** — odsprężona referencyjnie + sprzężona czasowo (publish-subscribe)
- **wspólna przestrzeń danych** — odsprężona referencyjnie i czasowo (krotki)

(Język Linda) przestrzeń krotek

Krotka (tuple) — rekord z kilku pól; przestrzeń = **zbiór wielokrotny** (multiset). 3 podstawowe operacje:

1. **in(t)** — usuń krotkę pasującą do wzorca t (blokująca)
2. **rd(t)** — pobierz **kopię** krotki pasującej do t (blokująca)
3. **out(t)** — dodaj krotkę t do przestrzeni

in i **rd** blokują, dopóki pasująca krotka się nie pojawi.

Nieblokujące wersje: **inp** (dla in) i **rdp** (dla rd).

Dopasowanie **asocjacyjne** po wzorcu (np. ("bob", "distsys", str)).

(Publish-subscribe) zdarzenia

Komunikacja przez **opisywanie zdarzeń**; subskrybent wybiera, co go interesuje. Elementy danych **nie** są jawnie identyfikowane (tylko dopasowywane).

- **tematyczne** (topic-based): pary (atrybut, wartość)
- **oparte na treści** (content-based): pary (atrybut, zakres), predykaty $\approx$ SQL

Magistrala zdarzeń (event bus) dopasowuje wydawców i subskrybentów. Middleware: przekazuje powiadomienie + dane, lub samo powiadomienie (subskrybent czyta). **Dzierżawa** (lease) — auto-usuwanie danych.

Zastosowanie: luźno powiązane, skalowalne systemy; ryzyko **wąskiego gardła** przy dopasowaniu (zwl. bezpieczeństwo/prywatność).

1.6 Architektury hybrydowe: chmura, edge/fog, blockchain

(Chmura obliczeniowa) 4 warstwy

Pula **zwirtualizowanych** zasobów, łatwa skalowalność, model **płatność za użycie** + SLA (Service Level Agreement). Warstwy:

1. **Sprzęt** (CPU, pamięć, dysk, łącze)
2. **Infrastruktura** — wirt. pamięć/serwery (Amazon S3, EC2)
3. **Platforma** — framework + storage (Azure, App Engine)
4. **Aplikacja** — apps dla użytkownika (Google Docs, Gmail)

Modele: **IaaS** (sprzęt+infra), **PaaS** (platforma), **SaaS** (aplikacje), **FaaS** (Function-as-a-Service, bez serwera). **S3** $\rightarrow$ pliki w **zasobnikach** (buckets).

(Edge computing i fog) brzeg / mgła

Napęd: IoT (Internet of Things) — niskie opóźnienia, nie wszystko do chmury.

- **edge computing (brzegowe)** — usługi „na brzegu” sieci, granica między siecią firmy a Internetem (ISP); ruch nie opuszcza sieci kampusowej (**infrastruktura brzegowa**)
- **fog (mgła obliczeniowa)** — pośrednia infrastruktura regionalna między brzegiem a chmurą; granice chmura/brzeg się zacierają

Warstwy: urządzenia (things) → edge → core Internet → cloud.

(Architektura blockchain) łańcuch bloków

= rozproszona księga (rejestr) transakcji. Założenie: uczestnikom **nie można ufać** ⇒ brak zaufanej strony trzeciej (banku).

Rola funkcji skrótu (hash): blok $B(i + 1)$ zawiera H danych $B(i)$; zmiana $B(i)$ wymaga przeliczenia skrótów **wszystkich** kolejnych bloków ⇒ **niezmiennność** (immutability), modyfikacja zawsze zauważalna. Niezmiennność ⇒ idealna **masowa replikacja**.

(Blockchain — walidatory i konsensus) rys. 3.12–3.13

Walidator dodaje blok do łańcucha. Kroki:

1. węzeł **rozgłasza** żądanie transakcji
2. walidator **grupuje** transakcje w blok, sprawdza ważność
3. rozgłasza **zwalidowany blok** do wszystkich

Kto idzie naprzód = **rozproszony konsensus**. 3 organizacje:

- **scentralizowana** — zaufana strona 3. (nie pasuje do idei)
- **z zezwoleniami** (permissioned) — mała grupa walidatorów; toleruje $k \leq (n - 1)/3$ złośliwych; $n <$ kilkudziesięciu
- **bez zezwoleń** (permissionless) — wszyscy walidują, **wybory lidera** (kosztowne ⇒ w praktyce centralizacja)

Zastosowanie: kryptowaluty (brak podwójnego wydania), tożsamość, dokumentacja medyczna.

(Tabela ewolucji obliczeń rozproszonych)

- **1960–79:** sieci rozproszone (ARPANET, zegary Lamporta)
- **1980–99:** klastry, gridy, MPI (Beowulf, SETI@home, Globus)
- **2000–09:** wirtualizacja, chmura, REST (VMware, AWS)
- **2010–19:** kontenery, mikrousługi, edge (Docker, K8s, fog)
- **2020–:** AI, serverless, quantum (Federated Learning, FaaS)

2 Blok 2: Programowanie równoległe, RPC, systemy plików

2.1 Programowanie równoległe

(Pamięć wspólna vs lokalna) dwa typy systemów

Klastry/gridy = maszyny z **pamięcią lokalną** połączone siecią, ale węzły to maszyny wielordzeniowe/wieloprocessorowe (**pamięć wspólna**).

- **Pamięć wspólna:** brak związku procesor–moduł pamięci; wszystkie moduły osiągalne dla wszystkich procesorów, *nie ma przesyłania danych*. Problem: jednoczesny dostęp do tych samych danych ⇒ kluczowa rola mechanizmów ochrony dostępu (**zamek**).
- **Pamięć lokalna** (sieci komputerowe): kluczowa rola **przesyłania komunikatów** między procesorami; własne przestrzenie adresowe; zadania = wiele procesów/programów.

(Synchronizacja: zamek i bariera) mutex / barrier

Zamek (*lock, mutex*) – chroni fragment kodu (**sekcję krytyczną**, np. dostęp do globalnych struktur / urządzeń peryferyjnych) przed wejściem > 1 wątku. Synchronizowany dostęp do sekcji krytycznej.

Bariera – wszystkie wątki muszą dojść do ustalonego miejsca, zanim którykolwiek pójdzie dalej. Synchronizacja: przejście do następnego etapu dopiero po zakończeniu poprzedniego przez *wszystkie* procesory. **Zamek pisarzy–czytelnicy** (*rw-lock*) – blokuje sekcję tylko gdy wątek *zmienia* dane; równoległe czytanie przez wiele wątków dozwolone.

(Ziarnistość (granularity)) Liczba operacji obliczeniowych między punktami synchronizacji/komunikacji.

- **Grube ziarno** (*coarse grain*) – dużo obliczeń, dużo danych na procesor, rzadka komunikacja ⇒ wysoka wydajność.
- **Drobne ziarno** (*fine grain*) – mało danych, częsta komunikacja/synchronizacja.

Cel wydajności: rzadkie używanie bariery; min. czasu kontroli dostępu (pam. wspólna); min. czasu komunikacji (pam. lokalna).

(Proces / wątek / serwer) **Proces** – program w wykonaniu + kontekst (segment instrukcji, danych, syst.). UNIX: powstaje przez `exec*` (zmienia plik w tym samym procesie) lub `fork`.

Wątek (*thread*) – instancja procedury, prywatne dane lokalne + dostęp do danych globalnych wg reguł zasięgu. Najwygodniejszy mechanizm na pam. wspólnej. Wątki *nie* biorą udziału w operacjach między maszynami. **Serwer** – udostępnia zasób klientom. **Demon** – serwer dostępny cały czas (start przy włączeniu komputera).

(Serwer bezstanowy vs stanowy) **Bezstanowy (stateless)** – nie przechowuje stanu klienta; np. serwer WWW. Po awarii po prostu wznowia pracę – brak procedur naprawczych.

Stan miękki – serwer utrzymuje stan, ale tylko przez ograniczony czas. **Stanowy (stateful)** – trwałe info o klientach; np. serwer plików z lokalnymi kopiami; trzeba jawnie usuwać stan; po awarii nie ma gwarancji najnowszych aktualizacji.

(Architektury oprogramowania)

- wielu klientów – jeden serwer (równoległe przetw. danych),
- jeden klient – wiele serwerów (*master-workers* / dawniej *master-slaves*); klient = koordynator; obl. numeryczne,
- sieć węzłów równouprawnionych (*peer-to-peer*) – połączenia logiczne sąsiadów.

(Wątki POSIX (Pthreads)) biblioteka `pthread_*`

Prefiks `pthread`; zwraca 0 = OK, inaczej kod błędu. `#include <pthread.h>`. (Bariery/rw-lock nie ma w standardzie C23!)

- `pthread_create(&id, &attr, start_routine, &arg)` – nowy wątek
- `pthread_self()`, `pthread_exit(&ptr)`
- `pthread_join(thr, **val)` – czeka na zakończenie (np. ESRCH)
- **zamek:** `PTHREAD_MUTEX_INITIALIZER`; `mutex_lock` (blok.), `trylock` (nieblok.), `unlock`
- **rw-lock:** `rdlock/tryrdlock` (czytelnicy), `wrlock/trywrlock` (pisarze)
- **bariera:** `barrier_init(&b, &attr, count)` (`count` = ile wątków ją łamie), `barrier_wait`

(MPI – podstawy) Message Passing Interface

Standard przesyłania komunikatów na maszynach z **pamięcią lokalną** (MPI Forum, poł. lat 90.). Biblioteka dla C/C++/Fortran. Zasada **SPMD** (Single Program Multiple Data). Uruchamianie: `mpirun -n <n> <prog>` lub `mpirun -c <n> (LAM)`.

- **Grupa procesów** – każdy proces ma indeks $0..n-1$.
- **Komunikator** – grupa + kontekst (np. topologia, kolory). Komunikat wysłany w danym kontekście odbierany tylko w tym samym. Uchwył `MPI_Comm`; domyślny `MPI_COMM_WORLD`.
- `MPI_Init(&argc, &argv)`, `MPI_Finalize()`; zwracają `MPI_SUCCESS`.

(Modele komunikacji MPI) 2 niezależne wymiary

1. Synchronizacja wzajemna:

- **synchroniczna** – nadawca czeka na gotowość odbiorcy i kończy gdy odbiorca potwierdzi odbiór (*handshake*).
- **asynchroniczna** – nadawca nie czeka na potwierdzenie (analogia: poczta elektroniczna).

2. Warunek powrotu z funkcji:

- **blokująca** – powrót po zakończeniu operacji (czeka na rezultat).
- **nieblokująca** – powrót natychmiast (nie czeka).

Możliwe dowolne kombinacje (np. nieblok.+synchr. wysyłanie + blok. odbieranie).

(MPI – procedury punkt-punkt) Send / Recv
MPI_Send(buf, count, datatype, dest, tag, comm) – blok., asynchr. (tag $\in 0..MPI_TAG_UB$).
MPI_Recv(buf, count, type, source, tag, comm, *status) – blok., asynchr.; MPI_ANY_SOURCE, MPI_ANY_TAG.
Warianty wysyłania: MPI_Bsend (asynchr. buforowane), MPI_Ssend (synchr.). Test nieblok.: MPI_Iprobe.
MPI_Sendrecv(...) – łączone wysyłanie+odbieranie, **chroni przed zakleszczeniem**.

(Blokada (deadlock) – przykład) Dwa równoprawne procesy robią Ssend (synchr.) do siebie nawzajem, potem Recv $\Rightarrow$ oba czekają na odbiór $\Rightarrow$ **blokada**. OK tylko dla producent-konsument.
Zalecenie ogólne: nadawca MPI_Send (asynchr.+blok.), długie $\rightarrow$ Bsend; odbiorca MPI_Recv po MPI_Iprobe; synchr. barierą MPI_Barrier.
Uwaga: nieblok. wysyłanie/odbiór wymaga *sprawdzenia* zakończenia transmisji przed zmianą danych – niskopoziomowe, raczej unikać.

(MPI – komunikacja grupowa)

- MPI_Bcast(buf, count, type, root, comm) – rozgłoszenie z *root* do grupy.
- MPI_Scatter – dzieli wektor i rozsyła podwektory.
- MPI_Gather – zbiera dane do root (odwrotność Scatter).
- MPI_Alltoall – każdy wysyła wszystkim.

Topologie wirtualne: adresowanie procesów we współrzędnych siatki (kartezjańska / grafu) zamiast przez *rank*; przypisane komunikatorowi.

2.2 RPC i gRPC

(RPC – zdalne wywołanie procedur) Remote Procedure Call
Wykonanie na innym komputerze procedury wywołanej lokalnie. Przekazywanie parametrów i wyników ukryte przed użytkownikiem (iluzja jednego programu). To dwa programy: **klient** (lokalnie) + **serwer** (zdalnie). Pełny program generowany automatycznie z modułów. Sam RPC *nie* zapewnia zrównoleglenia (trzeba np. wątków). **ONC RPC** (Sun RPC) – klasyczne, kryje się np. za NFS.
Składniki: protokoły+kodowanie (XDR), **język specyfikacji interfejsu (IDL)**, generator kodu **rpcgen**, biblioteka (filtry XDR), **rejestr procedur** (rpcbind/Solaris, portmap/Linux), rpcinfo.

(XDR) XDR (External Data Representation) – format danych zdef. na potrzeby RPC; **pełna przenośność** w środ. heterogenicznym. Konwersja pamięć $\rightarrow$ XDR = **szeregowanie**, odwrotnie = **deszeregowanie**, przez procedury zwane *filtrami*. Rejestr procedur: konwersja numeru programu RPC + wersji $\rightarrow$ numer portu.

(Język RPC (IDL) + rpcgen) Plik definicji protokołu (np. a.x, składnia jak C): obowiązkowa deklaracja programu serwera + ≥ 1 funkcji; typy złożone; stałe; komentarze.
rpcgen a.x generuje: a.h (nagłówki), a_xdr.c (filtry XDR), a_clnt.c (namiastki klienta), a_svc.c (namiastki serwera), a_client.c, a_server.c (szkielety), makefile.a. Biblioteka pozwala wybrać transport TCP/UDP.

(gRPC) nowoczesny RPC od Google
Wady starych RPC (CORBA): złożoność, TCP. Później XML-RPC (HTTP+XML), JSON-RPC. **gRPC** – pierwotnie projekt *Stubby* (Google), otwarty; transport **HTTP2**. Komunikacja w heterogenicznych systemach. IDL + szeregowanie = **Protocol Buffers** (*.proto); generator **protoc** tworzy szkielety serwera i klienta (C++, Java, Python, C#, Go, ...).
4 rodzaje metod zdalnych:

- **unarne** (simple) – 1 żądanie / 1 odpowiedź: rpc GetFeature(Point) returns (Feature)
- **server streaming** – returns (stream Feature)
- **client streaming** – rpc RecordRoute(stream Point) returns (RouteSummary)
- **bidirectional streaming** – rpc RouteChat(stream ..) returns (stream ..); strumienie niezależne, kolejność w każdym zachowana.

(Protocol Buffers (.proto)) IDL gRPC
Plik .proto definiuje typy danych + **kolejność szeregowania** (numery pól). syntax = "proto3";
message Number { float value = 1; }
service Calculator {
 rpc SquareRoot(Number) returns (Number){}
}
Numer „1” = kolejność szeregowania pola. Wektory: kwalifikator repeated (liczba elementów dowolna). Generacja (Python):
python -m grpc_tools.protoc -I. --python_out=.
--grpc_python_out=. calculator.proto
 $\Rightarrow$ calculator_pb2.py (klasy komunikatów), calculator_pb2_grpc.py (...Servicer serwer, ...Stub klient).

2.3 Rozproszone systemy plików

(Rozproszony system plików (DFS)) wymagania
Rozproszona implementacja modelu plików z podziałem czasu; wielu użytkowników współdzieli pliki i pamięć; model klient-serwer.
Wymagania: przezroczysty dostęp (jak do dysku lokalnego); jednora-zowe logowanie; odporność na awarie serwera; wydajność (zbliżona do lokalnego dysku); niezależność (różne OS); szyfrowanie danych w transmisji. **Replikacja** (powielanie) $\rightarrow$ wydajność + odporność + rozkład obciążeń.

(Współdzielenie i spójność) Poziomy współdzielenia danych:

- nieformalne przez interfejs klienta (NFS),
- rozproszone pliki (AFS, Coda),
- jednolity obraz w każdym węźle klastra (Lustre, Ceph, MFS, GFS).

Mechanizmy dostępu: wspólny odczyt bez zapisu / kontrolowany zapis (jeden modyfikuje) / jednoczesny zapis (wymaga synchronizacji).

Problem spójności (przy jednoczesnym zapisie, kopie podręczne $\Rightarrow$ wiele wersji):

- *stateless* (NFS) – klient okresowo sprawdza stan u serwera,
- *callback* (Lustre, Ceph, AFS, Coda) – serwer powiadamia klientów o zmianach (**obietnica oddzwonienia**, callback promise).

(Transakcje (klastrowe FS)) **Transakcja** = ciąg działań usługowych z gwarancją **niepodzielności** (*atomic*): kończy się pomyślnie albo (przy awarii) bez żadnych skutków.

- **serwer pojedynczej transakcji** – tylko węzeł główny zmienia strukturę FS; łatwiejszy, ale węzeł główny = wąskie gardło.
- **serwer wielokrotnej transakcji** – każdy węzeł wykonuje każdą transakcję; wymaga zarządzania blokadami i danymi.

(i-węzły i metadane) i-węzły (*inode*) – bloki informacyjne pliku, zawierają m.in. metadane.

Metadane („dane o danych”) – info o danych: rozmiar pliku, data utworzenia, typ, właściciel. **Mogą być obsługiwane oddzielnie** od kanału danych $\Rightarrow$ przekazane wyspecjalizowanemu serwerowi (klucz w Lustre).

(NFS) Network File System (SUN, 1985)

Prosty protokół klient/serwer; przezroczyste współdzielenie plików, emuluje interfejs UNIX. Moduły klienta i serwera w jądrze; zależność **symetryczna** (każda maszyna = klient i serwer). Przesyła tylko bloki, nie całe pliki ($\downarrow$ obciążenie sieci). Eksport katalogów (*broadcast/advertise*), montowanie u klienta; węzeł-klient *nie* może wtórnie eksportować.

Cechy: **bezzastanowy** (operacje **idempotentne**, powtarzalne); awaria serwera = zawieszenie, nie przerwanie; menadżer blokad (*lock manager*, v3); ACS/ACL; oparty na UDP+RPC. Korzysta z mechanizmu **RPC** (RPC/XDR). **Ograniczenia:** słaba skalowalność (liczbę klientów limituje wydajność serwera); zabezpieczenia słabe (nie dla sieci rozległych). Web-NFS = NFS przez Internet.

Architektura: klient (proces+klient NFS w jądrze + demon *biod*), serwer (serwer NFS w jądrze + *nfsd* + *mountd*).

(Lustre) Linux cLUSTRE – klastrowy FS

Linuxowy odpowiednik VAXClusters; skalowalny, bezpieczny, odporny, wysoka dostępność i przepustowość; klastry > 10 000 węzłów; otwarty (GNU). **Obiektowy FS**: dane na wielu urządzeniach, obiektami są katalogi i pliki. **Klucz: rozdzielenie metadanych od danych** ⇒ ogranicza wąskie gardło.

3 komponenty:

- **MDS** (MetaData Server) – struktura FS/metadane (bez danych); obsługa nazw, blokad (też podkatalogów); rejestr śledzi dane.
- **OSS** (Object Storage Server) – magazyny obiektów **OST** (Object Storage Targets); przechowuje dane jako obiekty (i-węzły).
- **klient Lustre** – gada z MDS (metadane) i bezpośrednio z OSS (dane); sterownik **LOV** (Logical Object Volume) – obiekt na wielu OST.

(Lustre – dostępność i niezawodność) Odporny na awarie OSS – przy awarii zachowuje się stabilnie (obiekty z uszkodzonego węzła niedostępne do naprawy); możliwe redundantne OSS. **Dwa MDS: aktywny + zapasowy** – zapasowy przejmie przy awarii. Konfiguracja/logowanie: **LDAP + XML**.

Cechy nowoczesnego klastrowego FS: jednolity obraz FS; wysoka zarządzalność; gwarancja spójności cache w dowolnej chwili; skalowalność (dynamiczna rekonfiguracja); odporność na awarię węzła; szybkie wykrywanie awarii sprzętu sieciowego; wydajność (równoważenie obciążeń).

2.4 Middleware i wirtualizacja

(Wrappery i brokery) komponenty pośredniczące

Wrapper (nakładka, adapter) – oferuje interfejs akceptowalny dla aplikacji klienckiej, przekształca jego funkcje na dostępne w komponencie. Rozszerzalność starszych systemów.

Bez middleware: każda z N aplikacji potrzebuje nakładki dla każdej innej ⇒ $N(N-1) = O(N^2)$ nakładek.

Broker (pośrednik) – scentralizowany komponent obsługujący wszystkie dostępy; aplikacja wysyła żądanie, broker kojarzy/łączy. ⇒ co najwyżej $2N = O(N)$ nakładek. **Middleware** ułatwia integrację systemów.

(Wirtualizacja) przenośność i izolacja

Wirtualizacja – rozszerzenie/zastąpienie istniejącego interfejsu, by naśladować zachowanie innego systemu (implementacja A na B). Zmniejsza różnorodność platform; **izolacja kodu** (ważne w chmurze); **przenośność** – najważniejszy powód (oddziela sprzęt od oprogramowania, przenosi całe środowisko). Geneza: lata 70., uruchamianie starego OS na mainframe IBM 370.

3 poziomy interfejsów: ISA (architektura zestawu instrukcji: uprzywilejowane / ogólne), wywołania systemowe (OS), API (biblioteki).

(Rodzaje wirtualizacji)

- **maszyna wirtualna procesu** – system uruchomieniowy / interpreter (JVM) lub emulacja (Win na Unix), jeden proces.
- **natywny monitor maszyny wirtualnej** (VMM) – na gołym sprzęcie; wiele OS gości jednocześnie; własne sterowniki.
- **hostowany monitor maszyny wirtualnej** – na zaufanym OS hosta; korzysta z jego funkcji; nowoczesne chmury.

Znaczenie: niezawodność i bezpieczeństwo (izolacja ⇒ awaria/atak nie dotyka całej maszyny).

(Kontenery (konteneryzacja)) przenośność + izolacja

Zbiór plików binarnych (obrazów) = środowisko aplikacji; współdzielą *ten sam OS* (różnica od VM), własne biblioteki/zależności. Lekkie, przenośne.

3 mechanizmy uniksowe (Linux):

- **przestrzenie nazw** (namespaces) – własny widok ID procesów itd.; np. PID (każdy kontener widzi własny init, PID=1); unshare --pid --fork --mount-proc bash.
- **unijny system plików** – warstwowe łączenie FS; tylko górna warstwa zapisywalna (= część kontenera); warstwy bazowe r/o (np. Ubuntu 20.04 + PHP 7.4).
- **cgroups** (grupy kontrolne) – ograniczenia zasobów (pamięć, CPU, priorytet) dla zbioru procesów ⇒ jeden kontener nie zagłodzi innych.

2.5 Klastry

(Klaster komputerowy) definicja

Grupa współpracujących, połączonych szybką dedykowaną siecią, **homogenicznych** (ta sama architektura procesora i ten sam OS), jednolicie zarządzanych maszyn, używanych jako pojedynczy system. **Klaster = rodzaj systemu rozproszonego**.

Węzły (node) – „pełne” maszyny (procesory, pamięć, I/O, OS). Dostęp z zewnątrz przez węzeł **dostępowy** (front-end); zarządzanie 1 stanowiskiem (KVM switch). Zwykle PC pod Linux/FreeBSD.

(Właściwości klastrów) cechy

1. **Przezroczystość** – użytkownik nie odróżnia klastra od pojedynczej maszyny (migracja procesów niewidoczna).
2. **Skalowalność** – dodawanie/usuwanie węzłów; dodanie węzła ⇒ liniowy wzrost wydajności.
3. **Rekonfigurowalność** – węzły dołączane/odłączane bez przerw ⇒ łatwy serwis.
4. **Niezawodność** – odporność na awarię pojedynczej maszyny; migracja procesów.
5. **Dostępność** – usługi dostępne mimo częściowych awarii (efekt skalowalności).
6. **Wydajność** – przez **równoważenie obciążeń** (load balancing).

SSI (Single System Image) – iluzja jednego komputera; **scheduler**; **migracja** procesów + **checkpoints** (punkty kontrolne) ⇒ wznowienie od ostatniego punktu. **heartbeats** – komunikaty kontrolne wykrywające awarie.

Słabość: klastry słabo nadają się do silnie powiązanych obliczeń z częstą komunikacją (lepiej: każdy węzeł niezależnie).

(Klastry vs SMP / NUMA / P2P) różnice

vs SMP (Symmetric Multi-Processor, np. wielordzeniowy PC):

- skalowalność: rozbudowa klastra = dodanie pełnego komputera (bez zmian); SMP wymaga przeróbek.
- dostępność: awaria 1 procesora w SMP ⇒ unieruchomienie całości; w klastrze nie przerywa pracy.
- zarządzanie: łatwiej zarządza się SMP (1 komputer dla OS).

vs NUMA – NUMA: wielordzeniowe węzły, lokalna szybka pamięć, jednolicie adresowana z pamięciami innych węzłów (dostęp zdalny wolniejszy); jeden OS ⇒ *nie* jest klastrem.

vs systemy rozproszone: klaster dąży do **anonimowości węzłów** (rozproszony – każdy element rozróżnialny); klaster używa komunikacji **P2P** (punkt-punkt, równoprawny dostęp), rozproszony – zwykle **klient-serwer** (hierarchia, master-slave).

(Typy klastrów) 4 podstawowe klasy

- **HPC** (High Performance Computing) – ↑moc obliczeniowa; jeden komputer równoległy; duże zadania jednego rodzaju; MPI; równoważenie; prace naukowe. Nowa generacja superkomputerów.
- **HA** (High Availability) – węzły powielają funkcje; awaria → automatyczne przejście (niewidoczne); cel = *nieprzerwana praca*, nie wydajność. Np. MOSIX (łączy z wydajnością).
- **HTC** (High Throughput Computing) – max liczba zadań w długim czasie (godz./dni/tyg.); zadania czasochłonne, węzły mogą się odłączać (restart na innym); maszyny heterogeniczne; związek z gridami.
- **LB** (Load Balancing) – rozdział ruchu na wiele węzłów; **Load Balancer** (sprzętowy / programowy) → najmniej obciążony węzeł; portale, serwery WWW. Np. LVS (Linux Virtual Server), F5, L4/L7.

Inne podziały: **aplikacyjne** (obl. równoległe) / **systemowe** (serwery, np. bazy); architektura: SSI, Beowulf, ad hoc.

(Modele organizacji klastrów)

- **Beowulz** – wirtualny superkomputer z tanich PC (Ethernet, Linux, MPI); 1 serwer + węzły-klienci; brak monitorów; jeden adres IP; NASA 1994. Wada: brak dzielonej pamięci dyskowej.
- **SSI** (Single System Image) – rozproszone zasoby jako jeden superkomputer; jednolity *root filesystem*; migracja + checkpoints. Wada: słaba skalowalność przy intensywnej wymianie info; patche jądra. Np. OpenSSI, Kerrighed, openMOSIX, z/VM, VMScluster.
- **klastry wsadowe** – systemy kolejkowania (PBS, OpenPBS/Torque, LSF/IBM); kolejki z parametrami: priorytet, max liczba zadań, limit na użytkownika/maszynę, limit pamięci; model klient-serwer.

shared data (dzielące pamięć stałą) – każdy węzeł dostęp do całego zasobu dyskowego; wymaga synchronizacji odwołań. **shared nothing** – węzeł widzi tylko swój fragment; żądania przekazywane do właściciela. **Współczesne:** Slurm, Kubernetes. **Fencing** (odgradzanie zasobów): hard fencing / STONITH.

(ROLA SYSTEMÓW ZARZĄDZANIA) po co?

Systemy zarządzania zasobami i planowania zadań (RM + scheduler) pozwalają wielu użytkownikom *współdzielić* klastry, superkomputer lub chmurę. Tworzą **jeden spójny interfejs** do wszystkich zasobów niezależnie od ich rozproszenia. Cele:

- efektywne wykorzystanie infrastruktury (CPU, RAM, węzły nie leżą bezczynnie),
- rozstrzyganie rywalizacji o zasoby (kolejka oczekujących prac),
- sprawiedliwy podział (fairshare, priorytety),
- automatyzacja, skalowalność, odporność na awarie.

Trzy menedżery obciążeń (Slurm): (1) przydziela użytkownikom dostęp do węzłów na czas, (2) daje ramy do uruchamiania/monitorowania zadań, (3) zarządza kolejką oczekujących prac.

(KOLEJKOWE vs ORKIESTRACJA KONTENERÓW) PBS/Slurm vs K8s
Systemy kolejkowe (PBS, Slurm) – HPC, zadania *wsadowe* (batch), MPI; użytkownik wstawia zadanie do kolejki, czeka na przydział, zadanie się kończy. Model **imperatywny**: skrypt opisuje kroki.
Orkiestracja kontenerów (Kubernetes) – usługi *długodziałające* (serwery, mikroserwisy), kontenery. Model **deklaratywny**: deklarujesz pożądany stan, system go utrzymuje (self-healing).

- kolejka: jednostka = *job*, kończy się; K8s: jednostka = *Pod*, podtrzymywany,
- kolejka: qsub/sbatch raz uruchamia; K8s: kontroler *ciagle* reaguje na awarie,
- wspólne: planowanie (scheduler), alokacja zasobów, monitoring, skalowanie.

3.1 PBS

(PBS — PODSTAWY) Portable Batch System

PBS (Portable Batch System) – system kolejkowania i dystrybucji obciążenia na Unix, heterogeniczne klastry, superkomputery. Powstał dla NASA. Wersje: *OpenPBS* (darmowa), *PBS Professional* (komercyjna). Rezerwuje i przydziela zasoby wg przyjętych zasad; **jeden spójny interfejs**.
Typy kolejek:

- **serial** – zadania w *jednym węźle*,
- **multi** – zadania *rozproszone* (np. MPI).

Identyfikator zadania: numer[.nazwa_serwera][@serwer] (np. 1556.csd.ia.pw.edu.pl).

(PBS — ZMIENNE ŚRODOWISKOWE) prefiks PBS_O_

Lista atrybutów (*variable_list attribute*) przekazywana do zadania. Wartości zmiennych z qsub (HOME, PATH, SHELL, MAIL) dostępne jako PBS_O_*:

- PBS_O_HOST – węzeł wywołania qsub,
- PBS_O_QUEUE – kolejka, do której wstawiono,
- PBS_O_WORKDIR – ścieżka robocza,
- PBS_NODEFILE – plik z listą węzłów (systemy równoległe/klas-trowe).

(PBS — POLECENIA) zarządzanie zadaniami

- qsub – wstawia zadanie, zwraca pełny identyfikator,
- qdel *id* – usuwa; qrerun *id* – wznowia,
- qhold *id* – wstrzymuje (wznowienie przez rerun),
- qalter *id* – zmienia atrybuty (np. -p priorytet),
- qmove *kol id* – przenosi do kolejki; qorder – zmiana kolejności,
- qstat – status zadań/kolejek/serwera,
- qselect – id zadań wg kryteriów (np. -u użytkownik, -r).

GUI: xpbs, monitor węzłów xpbsmon.

(PBS — OPCJE qsub i ALOKACJA) -l limit=war

Najpierw *oszacuj* zapotrzebowanie (liczba rdzeni, czas, pamięć). Opcje w linii lub w skrypcie (etykiety #PBS):

- -I interaktywnie; -q kolejka; -N nazwa (max 15 zn.),
- -S ściezka_int interpreter; -a data_czas start,
- -e/-o ścieżki stderr/stdout; -m kiedy mail (a przerwane, b start, e koniec, n brak),
- -v lista_zm (zm=wart,...); -V wszystkie zmienne.

Ograniczenia zasobów przez -l: nodes (liczba węzłów), cput (czas CPU wszystkich procesów), mem/vmem (pamięć), pcpu/pmem/pvmem (1 proces), walltime (czas rzeczywisty). Np. -l mem=10MB.

(PBS — qstat i URUCHAMIANIE) monitoring + skrypt
qstat: -n (z węzłami), -a (wszystkie), -au uz, -r (running), -f id (szczegóły), -q/-Q/-Qf kol (kolejki/limity), -B (serwer).

Sposób 1 (potok): echo ls -lt | qsub -q kol_moja

Sposób 2 (skrypt): qsub ./skrypt, opcje jako #PBS -q kol_moja.

Przykład MPI równoległy: #PBS -S /bin/tcsh

```
#PBS -N test ; #PBS -o test.log
```

```
#PBS -l nodes=6
```

```
#PBS -l walltime=12:00:00,cput=1:00:00
```

```
#PBS -l mem=30MB ; #PBS -m bae
```

```
cd $PBS_O_WORKDIR
```

```
NPROCS='wc -l < $PBS_NODEFILE'
```

```
mpirun -machinefile $PBS_NODEFILE -np $NPROCS a.out
```

3.2 Slurm

(SLURM — PODSTAWY i ARCHITEKTURA) slurmctld/slurmd

Slurm – open source, *odporny na awarie*, wysoce skalowalny menedżer klastrów. Nie wymaga modyfikacji jądra.

Architektura (demony):

- **slurmctld** – centralny demon w węźle zarządzania (planowanie, kolejka); opcjonalny zapasowy (failover),
- **slurmd** – w *każdym* węźle obliczeniowym; hierarchiczna, odporna komunikacja,
- **slurmdbd** – opcjonalna baza (księgowanie, fairshare).

Polecenia można uruchamiać z dowolnego miejsca w klastrze.

(SLURM — JEDNOSTKI ZARZĄDZANE) węzły/partycje/zadania/kroki

- **węzły (nodes)** – zasób obliczeniowy,
- **partycje (partitions)** = *kolejki*; grupują węzły w logiczne (nakładające się) zestawy; mają limity: rozmiar zadania, czas, uprawnieni użytkownicy,
- **zadania (jobs)** – alokacja zasobów na czas; uporządkowane wg **priorytetów**, przydzielane aż zasoby partycji się wyczerpią,
- **kroki zadania (job steps)** – (równoległe) podzadania w ramach alokacji; np. 1 krok na wszystkich węzłach albo kilka kroków na częściach alokacji.

(SLURM — POLECENIA) srun/sbatch/squeue/sinfo

- **srun** — uruchamia zadanie lub inicjuje kroki w czasie rzeczywistym; opcje zasobów (-N węzły, -n zadania, pamięć, cechy),
- **sbatch** — przesyła *skrypt* do późniejszego wykonania (zawiera srun),
- **salloc** — alokacja zasobów + powłoka (potem srun),
- **scancel** — anuluje zadanie/krok lub wysyła sygnał,
- **scontrol** — podgląd/modyfikacja stanu (admin/root),
- **squeue** — stan zadań/kroków (domyślnie running, potem pending wg priorytetu),
- **sinfo** — stan partycji i węzłów,
- **sprio** — składowe priorytetu; **sstat** — zasoby działającego zadania,
- **sacct** — księgowanie; **sbcast** — kopia pliku na lokalne dyski węzłów; **sview** — GUI.

(SLURM — MONITOROWANIE) sinfo / squeue
sinfo (PARTITION/AVAIL/TIMELIMIT/NODES/STATE/NODELIST):
*=domyślna partycja; stany *up/down/idle/alloc*; zwięzły zapis węzłów *adev[1-2]*.
squeue pole **ST**: R=Running, PD=Pending. **NODELIST(REASON)** — gdzie działa / czemu czeka: *Resources* (brak zasobów), *Priority* (kolejka za wyższym priorytetem).
scontrol show partition | node | job — szczegóły (np. PartitionName=debug TotalNodes=5 MaxTime, JobState=PENDING).

(SLURM — PRZYKŁADY i MPI) kroki + węzły
`srun -N3 -l /bin/hostname → 3 węzły, 1 zadanie/węzeł.`
`srun -n4 -l /bin/hostname → 4 zadania, 1 proc/zadanie.`
Skrypt z `#SBATCH --time=1: sbatch -n4 -w "adev[9-10]" -o my.stdout my.script` (opcje z linii nadpisują skrypt).
`salloc -N1024 bash, potem sbcast/srun/exit.`
MPI — 3 tryby: (1) Slurm uruchamia bezpośrednio przez *PMI2/PMIx* (nowoczesne MPI), (2) *mpirun* z infrastrukturą Slurm (starsze Open-MPI), (3) *mpirun* przez SSH/RSB (poza kontrolą Slurm; epilog + *pam_slurm_adopt*).

(SLURM — JOB ARRAYS (tablice zadań)) skalowanie
Tablice zadań — wydajne przesyłanie/zarządzanie kolekcją *wsadowych* zadań o identycznych wymaganiach (miliony w ms). Polecenia działają na całej tablicy lub na elementach. **Slurm tworzy 1 rekord** przy przesłaniu; kolejne dopiero przy zmianie stanu → bardzo skalowalne.
`sbatch --array=0-31, --array=1,3,5,7, --array=1-7:2 (krok), --array=0-15%4 (max 4 naraz). Min indeks 0, max = MaxArraySize-1 (domyślnie 1001).`
Zmienne: **SLURM_ARRAY_JOB_ID**, **SLURM_ARRAY_TASK_ID**, **_COUNT/_MAX/_MIN**. Identyfikacja elementu: `36_1`. Nazwy plików: `%A=JOB_ID, %a=TASK_ID`.
Zależności: `--depend=after | afterany | afterok | afternotok | aftercorr:id`.

3.3 Kubernetes

(K8s — KONCEPCJA i IaC) orkiestracja kontenerów
Kubernetes — orkiestrator kontenerów (open source, CNCF, inspirowany Google Borg). Automatyzuje wdrażanie, skalowanie, równoważenie i samonaprawianie kontenerów w środowiskach rozproszonych. Cechy: **auto-scaling** (horizontal pod autoscaling), **self-healing** (restart nieaktywnych kontenerów), przenośność.
Infrastruktura jako kod (IaC): pliki **manifestów** (YAML/JSON) opisują kluczowe aspekty (liczba replik, strategię wdrażania, monitoring) → pełna *traceability* i powtarzalność, brak ręcznej konfiguracji przez GUI.
Czego NIE robi: nie buduje obrazów (CI/CD poza K8s), nie narzuca middleware/baz/logowania, nie zarządza fizyczną infrastrukturą; nie jest pełnym PaaS.

(K8s — ARCHITEKTURA) control plane + worker nodes
Model **master-worker** (**control plane** + **węzły robocze**).
Control Plane („mózg”): globalne decyzje + utrzymanie pożądanego stanu:

- **kube-apiserver** — główny interfejs API (REST), *jedyny punkt kontaktu*, walidacja/autoryzacja; komunikacja przez **kubectl**,
- **etcd** — spójny magazyn klucz-wartość, *źródło prawdy* (stan + konfiguracja klastra),
- **kube-scheduler** — przypisuje nowe Pody do węzłów wg zasobów/polityk/ograniczeń,
- **kube-controller-manager** — uruchamia kontrolery (Node, Replication, Endpoints...).

Węzły robocze: **kubelet** (agent: pobiera spec Podów, dba o ich zdrowie, raportuje), **kube-proxy** (reguły sieciowe iptables/ipvs, load balancing), **Container Runtime** (containerd/CRI-O przez CRI).

(K8s — MODEL DEKLARATYWNY) desired state
Imperatywny: użytkownik wydaje serię kroków. **Deklaratywny (K8s):** *deklarujesz pożądaný stan* (spec), a system sam go osiąga i utrzymuje — nieustannie porównuje status (stan faktyczny) ze spec.
Zalety: stabilność/odporność (Pod padnie → kontroler wznowi), automatyzacja, **idempotentność** (wielokrotne zastosowanie = ten sam efekt), wersjonowanie (Git), łatwe skalowanie (zmiana 1 wartości).
Manifest: `apiVersion, kind (Pod/Deployment/Service), metadata` (nazwa/labels), `spec` (pożądaný stan), `status` (tylko do odczytu, zarządzany przez K8s).

(K8s — KONTROLERY i PĘTLA UZGADNIANIA) reconciliation loop
Kontrolery — „silniki” modelu deklaratywnego; procesy, z których każdy jest **pętlą sterowania** obserwującą stan przez API Server. Gdy wykryją rozbieżność spec vs status → działają, by doprowadzić do pożądanego stanu, i aktualizują status.
Pętla uzgadniania (reconciliation loop):
1. pobierz element z kolejki,
2. porównaj desired vs actual state,
3. wykonaj akcje korygujące,
4. aktualizuj status.
Przykłady: **Deployment Controller** (cykl życia, utrzymuje liczbę Podów), **ReplicaSet** (zadeklarowana liczba replik), **Node**, **Endpoints**, **Service Controller**.

(K8s — OBIEKTY i POJĘCIA) Pod/Deployment/Service

- **Kontener** — lekka, samodzielna jednostka z aplikacją i zależnościami,
- **Pod** — najmniejsza jednostka uruchomieniowa (1+ kontenerów na 1 węźle),
- **Deployment** — deklaratywne zarządzanie zestawem replik Podów (skalowanie, aktualizacje),
- **ReplicaSet** — utrzymuje liczbę replik; **StatefulSet** — aplikacje stanowe,
- **Service** — stały punkt dostępu + load balancing (ClusterIP/NodePort/LoadBalancer),
- **Namespace** — logiczna izolacja zasobów,
- **ConfigMap/Secret** — konfiguracja / dane poufne,
- **Job** — zadanie jednorazowe do końca; **CronJob** — wg harmonogramu (jak cron); **Ingress** (+ Kontroler Ingress) — routing HTTP/S.

(K8s — kubectl i PARAMETRY) uruchamianie
Definiowanie/uruchamianie przez manifest + **kubectl**:

- `kubectl apply -f deploy.yaml` — *kluczowa, deklaratywna*; tworzy lub wprowadza tylko konieczne zmiany (idempotentna),
- `kubectl get pod|deployments|services` (-o wide, -l app=nginx),
- `kubectl describe pod` — events/conditions (debug),
- `kubectl logs <pod> -f, kubectl top pods`.

Parametry zadań (Deployment spec): `replicas, containers.image, resources (limits CPU/RAM), strategy.type (RollingUpdate/Recreate), maxSurge/maxUnavailable, ports, volumes`. **Parametryzacja/zmienne** przez **ConfigMap** i **Secret**.

(K8s — MONITORING i OBSERVABILITY) **Metryki:** z **kubelet** (CPU/RAM Podów, sieć/dyski, restarty), z **API Server** (liczba żądań, latencja, błędy autoryzacji). **Logi:** kontener → stdout/stderr, kubelet zbiera, agregacja *ELK/Fluentd*. **Events i stan** przez API. **Narzędzia:** *Prometheus* (eksport metryk, alerting) + *Grafana*; **Health checks:** *Liveness/Readiness/Startup probes*.

(INTERAKCJA UŻYTKOWNIKA (K8s)) (1) użytkownik deklaruje stan przez **kubectl** → **kube-apiserver**; (2) **apiserver** waliduje i zapisuje w **etcd**; (3) **kube-scheduler** wybiera węzeł dla nowego Poda; (4) **controller-manager** pilnuje pożądanego stanu (węzeł padł → Node Controller); (5-7) **kubelet** wykrywa Pod, instruuje **Container Runtime**, raportuje stan; (8) **kube-proxy** konfiguruje sieć i Serwisy. Spójność dzięki centralnemu API Server.

4 Blok 3: Gridy obliczeniowe (HTCondor, DIRAC)

4.1 Gridy — podstawy

(Czym jest GRID) idea

Grid = środowisko integrujące **rozproszone, heterogeniczne** zasoby (jednostki obliczeniowe, klastry, superkomputery, pamięć dyskowa/taśmowa, bazy danych, aparatura) w jedną logiczną całość. Powstał w latach 90. z idei *The Grid* (analogia do Internetu — sieci łączącej całość zasobów świata). Trzy (częściowo sprzeczne) definicje:

- infrastruktura sprz.+progr. dająca **niezawodny, spójny, tani, wszechobecny** dostęp do mocy obl.;
- mechanizm **kontrolowanego współdzielenia** zasobów w dynamicznych, skalowalnych **organizacjach wirtualnych**;
- zbiór **luźno powiązanych, rozproszonych geogr., heterogenicznych** zasobów (def. najogólniejsza).

Cel: wykorzystanie zasobów na odległość. Grid $\neq$ klaster (klaster = jednolicie administrowany, do obliczeń **równoległych**; grid = interfejs między częściami systemu, niska wydajność łączy).

(Zastosowania gridów) kto

Użytkownicy: głównie **naukowcy** (fizyka wysokich energii — CERN, astronomia). Typowe zadania:

1. dostarczanie **mocy obliczeniowej** (płacisz udostępniając własne maszyny gdy beczynne);
2. **współpraca naukowa** — organizacje wirtualne (fizyka, kosmologia, chemia kwant., biologia mol.);
3. zdalne **eksperymenty/obserwacje** drogą aparaturą (synchrotron, mikroskopy, teleskopy) i **analiza danych**;
4. jednolita infrastruktura dla wielu biur projektowych/laboratoriów (np. testy nowych leków, pojazdów);
5. **zdecentralizowana** kontrola/sterowanie systemami (energet., telekom.) — odporność na awarie pojedynczego węzła.

(Wymagania (cechy) gridów) cechy

- **Pojedyncze logowanie** — jeden raz potwierdzasz tożsamość, dostęp do zasobów wg praw; PKI praktycznie niezastąpiona (certyfikat → automatyczny dostęp).
- **Automatyczne szeregowanie** procesów i przesyłanie danych — system sam wybiera miejsce wykonania (gdzie jest oprogramowanie + wolne zasoby), dane wędrują automatycznie przy migracji.
- Otwarte, **standardowe protokoły i interfejsy** — interoperacyjność wielu gridów.
- Środowisko **heterogeniczne, ale wirtualnie homogeniczne** — jednolity ogłąd zasobów.
- **Skalowalność** (tysiące węzłów/użytkowników nie psują pracy), **odporność na awarie, wysokie bezpieczeństwo**.
- **Swoboda przyłączania/odłączania** zasobów (struktura dynamiczna), **autonomia ośrodków, łatwy interfejs**.
- **Migracje** — programy mogą się przemieszczać między węzłami; migracje wydajnościowe rzadkie (drogie); ważna detekcja awarii + restart na innym węźle (najlepiej od „kroku milowego” / checkpointu).
- Kompatybilność/bezawaryjność (własny komplet bibliotek), minimalne ograniczenia narzędzi.

(Organizacje wirtualne (VO)) vo

VO = zbiór osób i instytucji współdzielących zasoby gridu dla **wspólnego celu**. Ortogonalne do realnej struktury instytucji — rozwiązują konflikt: pełna skalowalność + autonomia ośrodków vs pojedyncze logowanie. Zamiast kont na każdym zasobie → konto skojarzone z VO. Przypisanie użytkownika do VO niesie info o jego uprawnieniach. Użytkownik może należeć do **wielu VO** (przy wysyłaniu zadania trzeba podać nazwę VO — samo zalogowanie nie wystarczy). Pozwala różnicować priorytety (praca: normalny; szkolenie: niski, wyniki kasowane po czasie).

(Pliki w gridach — zarządzanie danymi) pliki

Dwa typy plików:

- **Pliki lokalne (zwykle)** — małe (bajty-MB), na potrzeby zadania, słane z zadaniem, kasowane po odebraniu wyników; magazyny przejściowe (wielogigabajtowe).
- **Dane masowe** — przechowywane przez grid (TB/PB), często *istota użyteczności* gridu (analiza ogromnych danych). Na **Storage Resources** (dysk/taśma), dostęp przez API.

Federacja = łączenie wielu plików w jedną logiczną całość (gdy dane > pojemność centrum). **Replikacja** = te same dane w wielu miejscach (gwarancja identyczności). Replikacja → wydajność (wybór najbliższej repliki, brak konkurencji), repliki automatyczne / **cache**. Pliki w gridzie traktowane jak hurtownia danych: **nigdy nie zmieniane** (tylko zapis/odczyt/kasowanie) — brak systemu propagacji zmian; przestarzałe → skasuj wszystkie repliki i utwórz nowy plik.

(Uruchamianie wielu zadań) wiele

Jednostka pracy = **zadanie (job)** = program + dane wej. + parametry.

- **Job arrays (zadania tablicowe)** — jednym poleceniem setki/tysiące **identycznych** kopii programu, każda z unikalnym **indeksem** (zmienna środowiskowa) wybierającym dane (dane_1.txt, dane_2.txt). Funkcja workload managerów / job schedulerów. Zapewnia powtarzalność, eliminuje ręczne skrypty.
- **Workflows (przepływy pracy)** — fundament: **DAG** (skierowany graf acykliczny). Reprezentują **zależności** między zadaniami (C startuje po A i B). Brak cykli ⇒ jednoznaczna kolejność. Kluczowy **staging** danych (auto-kopiowanie na węzeł + zapis wyników). Wyniki muszą być jednoznacznie identyfikowalne (nazwy z ID zadania/etapu).

Automatyzacja + niezawodność: system wykrywa nieudane uruchomienia, ponawia / oznacza jako błędne.

(Architektura warstwowa (Foster/OGSA)) arch

4 warstwy (od dołu):

1. **Fabric (sieć szkieletowa)** — interfejsy do zasobów lokalnych, zapytania o stan, zarządzanie/blokowanie.
2. **Connectivity (łączności)** — protokoły komunikacji + zabezpieczeń (uwierzytelnianie); **delegacja uprawnień** użytkownika do programów (uwierzytelniane są często programy, nie użytkownicy).
3. **Resource (zasobów)** — zarządzanie pojedynczym zasobem (info, sterowanie dostępem).
4. **Collective (zbiorcza)** — wiele zasobów: odnajdywanie, alokacja, planowanie, replikacja.

+ **Applications (aplikacji)**. Warstwy collective/connectivity/resource = **middleware (warstwa pośrednicząca)**. Trend: **architektura zorientowana na usługi (SOA)** → **OGSA** (Open Grid Services Architecture).

(Standaryzacja middleware: **UMD vs OSG**) **UMD** (Unified Middleware Distribution, EGI, Europa): integruje ARC, HTCondor, UNICORE; AAI przez IGTF; dane: **dCache**, **FTS**; proces QA.

OSG Software Stack (Open Science Grid, USA): **dHTC** (distributed High-Throughput Computing), ekosystem **HTCondor + HTCondor-CE**; dane: **OSDF/XRootD** (caching zamiast statycznej replikacji); autoryzacja **SciTokens** zastępuje X.509. Współpracują w **WLCG** (CERN).

(Języki opisu zadań (**JDL, JSDL**)) **JDL** (Job Description Language) — wywodzi się z **ClassAds** Condora, prosta składnia (Type, Executable, Arguments, InputSandbox, VirtualOrganisation, requirements, rank), wygodny do ręcznej edycji; obsługuje zadania złożone (DAG).

JSDL (Job Submission Description Language) — oparty na **XML**, dobry do auto-generacji; opisuje tylko **pojedyncze** zadanie (brak zadań złożonych/parametrycznych — świadoma decyzja), uniwersalny i zstandaryzowany.

(Atuty: ARC vs UNICORE) platformy
UNICORE (Uniform Interface to Computing Resources; Niemcy, FZ Jülich; HPC):

- SOA + Web Services (WS-Agreement, OGSA); 3 warstwy: **Client** (Rich Client/portale/CLI), **Gateway & Services** (UNICORE/X — pojedynczy punkt wejścia, stan zadań, workflow), **TSI** (Target System Interface — tłumaczy na Slurm/PBS).
- Atuty: **zaawansowany silnik workflow**, bezpieczeństwo klasy enterprise (PKI, X.509, SAML, SSO), **przezroczystość** (ukrywa różnice OS). Komponent PRACE/EGI.

ARC (Advanced Resource Connector / NorduGrid; 2001, Skandynawia):

- Filozofia: **minimalizm + decentralizacja**, brak centralnych serwerów. Dwa filary: **Smart Client** (cała logika cyklu życia po stronie klienta) + **A-REX** (ARC Resource Execution Service — przyjmuje opisy ADL/xRSL, pobiera pliki, przekazuje do Slurm/HTCondor).
- Atuty: wysoka **skalowalność + odporność** (brak SPOF), wbudowany **data caching**, prostota wdrożenia. Filar **WLCG** (ATLAS, CMS).

(Inne platformy/projekty) **BOINC** — volunteer computing (VC), klient-serwer, HTC; komputery anonimowe/niezaufane/nieprzewidywalne; **SETI@Home** (1999–2020), Folding@home, Einstein@home. **WLCG** (2002, ~300 centrów, 42 kraje, do 3 mln zadań/dzień; middleware: **DIRAC**). **OSG**, **EGI** (2010), **ESGF**, **PRACE**. Polska: **PLGrid** (ICM, PCSS, Cyfronet AGH, WCSS, CI TASK, NCBJ).

4.2 HTCondor

(HTCondor — idea) htidea

HTCondor = middleware do **HTC (High-Throughput Computing)** — duża liczba zadań przez **długi** czas (tygodnie/miesiące), a nie szybkość pojedynczego (to HPC). Warstwa pośrednia użytkownik–infrastruktura. Historia: **Condor** (1988, U. Wisconsin–Madison) → **Condor-G** (integracja z Globus Toolkit, dostęp do gridów) → **HTCondor** (2012).

Korzenie: **cycle-scavenging** — „kradzież” nieużywanych cykli procesora bezczynnych stacji roboczych = **obliczenia oportunistyczne** (zasoby dostępne sporadycznie, mogą być odebrane w każdej chwili). Dziś też: klastry dedykowane, chmura, GPU, gridy (przez **HTCondor-CE** — Compute Entrypoint, brama do innych systemów wsadowych).

(Kluczowe cechy HTCondor) **Wywłaszczanie (preemption)** — suspend lub migrate zadania, gdy właściciel maszyny wraca lub pojawia się zadanie o wyższym priorytecie. **Checkpointing** — migawki stanu (w niektórych universe); po wywłaszczeniu wznowienie od ostatniego checkpointu na innej maszynie. **ClassAds** — elastyczne dopasowywanie. **Fair-share scheduling**. Zakaz wieloprocusowości w aplikacjach (fork/exec, IPC, wątki jądra, współdzielone pliki R/W).

(Demony HTCondor) demony

- **Central Manager** (serce puli):
 - condor_collector — zbiera ClassAdy (oferty) wszystkich maszyn i zadań; globalny widok puli.
 - condor_negotiator — **matchmaking** (dopasowanie zadań do zasobów) wg polityk.
- **Access Point / Submit Machine** — condor_schedd: zarządza kolejką zadań, uruchamia condor_shadow dla każdego zadania.
- **Execution Point / Execution Machine** — condor_startd: reklamuje atrybuty maszyny (pamięć, rdzenie, OS) do kolektora; condor_starter: faktycznie uruchamia i monitoruje zadanie.
- Wspólny: condor_master — pilnuje pozostałych demonów, restartuje, sprawdza nowe binaria, powiadamia admina.

(ClassAds i model matchmaking) classads

ClassAds = pary atrybut–wartość opisujące maszyny i zadania (jak ogłoszenia drobne). Zadanie podaje wymagania (requirements) i preferencje (rank), maszyna reklamuje możliwości. condor_negotiator szuka optymalnego dopasowania. Pozwala rozróżnić zasoby dedykowane vs oportunistyczne, zarządzać GPU/specjalistycznymi procesorami. Każde zadanie = **Job ClassAd** tworzony przez condor_submit.

(Universe w HTCondor) universe

Universe = środowisko wykonawcze dla zadania (w pliku submit; domyślnie vanilla):

- vanilla — domyślny, najmniej ograniczeń, wspiera pliki, **bez** checkpointingu;
- standard — checkpointing + zdalne wywołania systemowe (RPC do submit machine);
- grid — wysyłanie zadań do zasobów gridowych przez interfejs HTCondor;
- java — zadania dla JVM;
- scheduler — lekkie zadania duplikowane jako demon na hoście submit;
- local — uruchamianie lokalnie na submit machine;
- parallel — programy wymagające **wielu maszyn** do jednego zadania (np. MPI);
- vm — zadanie jako obraz dysku (maszyna wirtualna);
- container / docker — zadanie jako obraz kontenera (Docker/Singularity).

(Plik submit i polecenie queue) submit

Plik **submit (.sub)** — tekstowy opis sterujący przesyłaniem. Kroki: **dekompozycja** pracy na zadania (czas w minutach/godzinach — nie za krótkie/długie) → przygotowanie wsadowe → plik opisu → condor_submit → ID = **Cluster.Proc**.

Typowe pola: executable, arguments, output, error, log, request_cpus, request_memory, request_disk, should_transfer_files=yes, when_to_transfer_output=on_exit, transfer_input_files.

queue — instrukcja kończąca opis: wyślij zadanie do kolejki. queue 150 = **150 instancji** (job array) jednym poleceniem. **Sandbox** = pliki zadania: input (przed), output (wynikowe), scratch (w trakcie).

(Zmienne automatyczne (job arrays)) \$(Cluster)/\$(ClusterId) — wspólny dla zestawu (atrybut ClusterId). \$(Process)/\$(ProcId) — unikalny dla każdej instancji (0,1,...). Przykład: arguments = input_file.\$(Process), output = out.\$(Process), error = err.\$(Process). Inne: \$(ARCH), \$(OPSYS), \$(SUBMIT_FILE), \$(SUBMIT_TIME).

(Identyfikacja / monitorowanie / kontrola) condor_submit (wyślij) → ID Cluster.Proc. condor_q (monitor kolejki). condor_ssh_to_job (połącz się z działającym zadaniem). condor_rm (usuń z kolejki). condor_history (zadania zakończone — niewidoczne dla condor_q). Plik log = dziennik zmian stanu (start, zużycie, koniec, wywłaszczenie, kod zakończenia).

(Workflows: DAGMan) dagman

DAGMan = narzędzie HTCondor do przepływów pracy jako **DAG**. Auto-przesyłanie zadań w ustalonej kolejności (zależności). Plik opisu DAG:

- JOB NodeName SubmitDescription [DIR dir] [NOOP] [DONE] — mapuje węzeł na plik submit; NodeName unikalny, rozróżnia wielkość liter, bez spacji/znaków . & +.
- Węzeł = **lista zadań** + opcjonalny **PRE-script** (walidacja wej.) + opcjonalny **POST-script** (weryfikacja wyj., czyszczenie).
- PARENT A CHILD B — zależność (A musi się zakończyć **pomyślnie** przed B); wiele rodziców/dzieci: PARENT p1 p2 CHILD c1 c2.

Przykład **Diamond DAG**: PARENT A CHILD B C + PARENT B C CHILD D. **Rescue DAG** (DONE) przy niepowodzeniu.

4.3 DIRAC

(DIRAC — idea) diridea

DIRAC (Distributed Infrastructure with Remote Agent Control) — zaawansowany **middleware** / „**interware**”: warstwa pośrednia integrująca różne technologie (gridy, chmury, lokalne klastry, HPC) w spójną całość. Powstał ~2002 dla **LHCb** (Monte Carlo), dziś uniwersalny (m.in. wewnątrz WLCG). Tworzy iluzję **pojedynczego, dużego komputera** — dostęp transparentny i scentralizowany. Modułowy, rozszerzalny; oparty na **agentach** działających autonomicznie. Obsługuje setki tys. jednocześnie zadań i setki ośrodków. Zasoby: EGI/OSG/NDGF, OpenStack / Amazon EC2, klastry/HPC. Następca: **DiracX** (od 2023).

(WMS i pilot jobs (model pull)) wms

WMS (Workload Management System) — przydziela i wykonuje zadania, **wspólna kolejka** dla zadań użytkownika i produkcyjnych → globalne polityki priorytetów.

Model „pilot jobs”: **piloty** = zadania-wrappery (nakładki) uruchamiane na węzłach (workers); po konfiguracji środowiska **pobierają** właściwe zadania z centralnej kolejki DIRAC. To strategia **pull** (zasoby zgłaszają gotowość i pobierają) zamiast **push** (przydział centralny). Zalety: lepsze wykorzystanie zasobów, dynamiczna reakcja na obciążenie, mniej błędów z niedopasowania.

Komponenty WMS: Job Manager, TaskQueue DB + Director, Matcher, Job DB, Job State Update, Job Monitoring; na węźle pilot: Watchdog + Job Wrapper + Application.

(DMS — zarządzanie danymi) dms

DMS (Data Management System) — duże zbiory danych rozproszone geograf. Dane na **Storage Elements (SE)**, system utrzymuje repliki. **File Catalog** = lokalizacja + metadane plików:

- **LFC** (LCG File Catalog) — zewnętrzny, tradycyjny (projekt LCG, CERN);
- **DFC** (DIRAC File Catalog) — nowszy, zintegrowany, + metadane.

Jednolita **przestrzeń nazw (LFN)** — dane jako spójny system plików niezależnie od fizycznej lokalizacji (PFN). Operacje: przesyłanie, replikacja, usuwanie zbędnych, kontrola integralności, zapytania po metadanych. **Transformation Service** — **automatyczne przetwarzanie**: „weź dane → wykonaj operację → zapisz wynik”, warstwa między File Catalog/WMS a zasobami.

(Bezpieczeństwo w DIRAC) secdir

Możliwość wykonywania operacji **w imieniu użytkownika** → mechanizmy **delegacji**:

- **Certyfikat proxy** — tymczasowy certyfikat na bazie certyfikatu użytkownika, nowa para kluczy, **ograniczony czas ważności**; pozwala systemowi działać bez ujawniania głównego klucza prywatnego.
- Uwierzytelnianie w usługach gridu / **tokeny**; **SSO** (Single Sign-On).
- dirac-proxy-init. Wspólny system autoryzacji/konfiguracji — **DIRAC Configuration System** (API/CLI/Web widzą zadanie identycznie).

(Interfejsy, JDL, przesyłanie zadań) dirjdl

Opis zadania w **JDL** (Executable, Arguments, InputSandbox, OutputSandbox, StdOutput, CPUTime). **Sandbox (piaskownica)** — przesyłanie **małych** plików wej./wyników bezpośrednio; większe dane → SE.

Interfejsy: **CLI** (dirac-wms-job-submit/status/kill/get-output, dirac-dms-..., dirac-proxy-init); **API Python** (klasy Dirac — submitJob(), Job — setExecutable); **Web Portal** (Job Monitor, File Catalog Browser, Accounting/Monitoring).

Cykl: dirac-wms-job-submit job.jdl → **JobID**; dirac-wms-job-status <JobID> (Status/MinorStatus/Site); dirac-wms-job-get-output <JobID>.

(Dane w zadaniach (DMS — polecenia)) dirac-dms-add-file <LFN> <FILE PATH> <SE> [GUID] — wyślij do SE i zarejestruj w katalogu. dirac-dms-get-file <LFN> — pobierz. dirac-dms-replicate-lfn <LFN> <SE> — replika; dirac-dms-lfn-replicas <LFN> — lista replik; dirac-dms-show-se-status — Storage Elements. W JDL: InputData, OutputData, OutputSE. Konwencja LFN: /<vo>/user/<initial>/<username>/ + ID zadania (unikaj kolizji nazw).

(Zadania parametryczne (job arrays)) param

Zadanie parametryczne = jeden submit → zestaw zadań przez parametry. W JDL atrybut Parameters:

- **lista** (stringi/liczby): Parameters = {"first","second",...}; symbol %s zastępowany wartością (Arguments="%s", StdOutput="StdOut_%s");
- **liczba** + ParameterStart + ParameterStep (np. 20 liczb od 1 do 2); indeks %n.

Przesyłane jako zwykłe zadania → lista JobID (JobID = [1047,1048,...]). Wyniki: dirac-wms-job-get-output 1047 1048

(Workflows w DIRAC) Przepływ pracy DIRAC = sposób opisu zadań (**każde** zadanie z API jest workflow). Reprezentacja: **XML** (+ Python). dirac-jobexec jobDefinition.xml parsuje plik. Struktura: **Workflow** ⊃ **Step (krok)** ⊃ **Module (moduł)**, każdy z parametrami; kroki/moduły uporządkowane. **Tylko kolejność liniowa (bez DAG-ów** — w odróżnieniu od HTCondor DAGMan). Brak własnej bazy/usługi/agenta. Skalowanie i elastyczne dopasowanie do heterogenicznych środowisk dzięki pilotom + modułowości.

5.1 Blok 4. Przetwarzanie w chmurze

(Definicja chmury) cloud computing

Chmura = model *biznesowy* udostępniania zasobów obliczeniowych (sprzęt, oprogramowanie) przez internet **na żądanie**, w modelu **pay-per-use**. Ewolucja gridów: ten sam cel (zasoby przez sieć), ale grid bez modelu płatności. Przejście IT z *produktu* na *usługę* (od ok. 2007). Wszystko traktowane jak usługa (*as a service*), regulowana umową **SLA** (service-level agreement). Wyrosła na bazie: wirtualizacji, procesorów wielordzeniowych, SOA/usług sieciowych, gridów, automatyzacji DC.

(5 cech NIST) pięć postulatów

1. **On-demand self-service** (samoobsługa na żądanie): klient sam, automatycznie, niemal natychmiast zamawia zasoby bez angażowania personelu dostawcy.
2. **Broad network access** (powszechny dostęp przez sieć): dostęp przez internet z dowolnego urządzenia (PC, tablet, smartfon); niezależność od lokalizacji.
3. **Resource pooling & multitenancy** (pula zasobów / wielonajemność): zasoby fiz. i wirt. współdzielone między wielu klientów, przydzielane/zwalniane dynamicznie.
4. **Rapid elasticity** (elastyczność): skalowanie w górę/dół niemal natychmiast ⇒ wrażenie nieograniczonych zasobów.
5. **Measured service** (rozliczanie wg wykorzystania): pay-per-use, pomiar zużycia (czas serwera, ilość danych) ⇒ transparentność.

(Modele usługowe IaaS/PaaS/SaaS) poziom abstrakcji

Hierarchia wg poziomu abstrakcji i odpowiedzialności dostawcy:

- **IaaS** (Infrastructure): surowe zasoby (CPU, RAM, dysk, sieć) jako VM. Klient steruje OS, aplikacjami; brak kontroli nad sprzętem. Rozlicz. wg czasu (EC2 godzinowo, Azure minutowo). *Przykłady*: AWS EC2, Google Compute Engine, Azure.
- **PaaS** (Platform): platforma + narzędzia (języki, biblioteki, serwisy) do wdrażania własnych aplikacji bez instalacji oprogramowania serwerowego. Klient zarządza tylko aplikacjami. Wada: brak przenośności (lock-in). *Przykłady*: Google App Engine, Azure App Service, AWS Elastic Beanstalk, OpenShift, Heroku.
- **SaaS** (Software): gotowa aplikacja przez przeglądarkę/API. Klient nic nie zarządza. Wada: brak personalizacji kodu. *Przykłady*: M365, Google Workspace, Salesforce, Dropbox.

Inne aaS: FaaS (function), CaaS (container), WaaS (workload), DBaaS, Security aaS → zbiorczo **XaaS**.

(Modele wdrożeniowe) deployment

NIST: prywatna, publiczna, społecznościowa, hybrydowa.

- **Prywatna**: jedna organizacja, pełna kontrola; sektor finansowy / dane wrażliwe.
- **Publiczna**: dla ogółu, współdzielona (multitenant), rozliczana wg użycia. (EC2, Azure).
- **Społecznościowa**: kilka organizacji o wspólnych wymaganiach (np. rządowe, zdrowie).
- **Hybrydowa**: lokalna/prywatna *połączona* z publiczną; infrastruktury odrębne, ale współpracujące (orkiestrowane); dane krytyczne → prywatna, reszta → publiczna.
- **VPC** (Virtual Private Cloud): wydzielona, logicznie odseparowana część chmury publicznej, łączona VPN-em.

Multi-cloud: korzystanie z usług *wielu* dostawców; w odróżnieniu od hybrydowej chmury *nie* są zwykle połączone – różne komponenty u różnych dostawców (obliczenia AWS, sieć Azure). Cel: **best-of-breed**, optymalizacja kosztów, odporność (Disaster Recovery), unikanie **vendor lock-in**.

(Chmura vs pokrewne) **Outsourcing IT**: zasoby stałe, umowy długoterminowe (vs pay-per-use). **ASP**: wynajem gotowych aplikacji (poprzednik SaaS). **Grid**: współdzielone zasoby badawcze, wkład projektowy (vs model biznesowy chmury).

(Rynek, role, płatności, FinOps) ekosystem

Uczestnicy: klient (B2B negocjuje SLA / B2C ujednolicono SLA, freemium), dostawca (AWS, IBM, Google, Azure, SAP), operator DC, audytor (np. ISO/IEC 27018), broker.

Modele rozliczeń: *on-demand / pay-as-you-go* (max elastyczność, też pay-per-execution/serverless np. Lambda); *rezerwacje* (reserved/CUD, opłacalne gdy wykorzystanie >60–70% czasu); *spot* (nadwyżki, do –90%, odbierane w dowolnej chwili → batch); *Free Tier*.

FinOps (Financial Operations): optymalizacja kosztów chmury – monitoring (Cost Explorer), budżety/alerty, etykiety, auto-odłączanie zasobów; unikanie **overprovisioningu** (zbyt drogę zasoby).

5.2 Stos chmurowy i odpowiedzialność

(Wirtualizacja i konteneryzacja) źródło wykorzystania zasobów

Wirtualizacja: logiczny podział fiz. zasobów IT na wirtualne (VM); ukrywa sprzęt, prezentuje abstrakcję. VM = OS + aplikacja + zależności w jednej jednostce; łatwa migracja między serwerami. Zarządzana przez **hiperwizor** (Xen, KVM, Hyper-V, VMware).

Kontenery: lekka wirtualizacja na poziomie OS – współdzieli jądro hosta ⇒ mniejsze, szybszy start, większa gęstość upakowania, dobra izolacja, idealne dla mikrousług. Orkiestracja: **Kubernetes**, Docker Swarm.

Różnica: VM wirtualizuje *sprzęt* (cały serwer), kontener wirtualizuje OS (wspólne jądro).

(Stos chmurowy (cloud stack)) warstwy Buyya

Od dołu do góry (odzwierciedla IaaS/PaaS/SaaS):

- **System Infrastructure** (CPD): procesory wielordzeniowe, magazyny, sieć.
- **Virtualization & containerization**; serwery/sieci wirtualne (SDN).
- **VM instances** + zarządzanie/wdrażanie VM. ← *core middleware* (IaaS).
- **Guest OS**, usługi development/deployment.
- **Cloud Hosting Platforms**: QoS, admission control, SLA mgmt, metering, accounting.
- **User-level middleware**: Web 2.0, workflows, biblioteki (PaaS).
- **SOA / Web services**, szerokopasmowa sieć/internet, otwarte/REST API.
- **Cloud applications & users** (SaaS).

Przekrojowo: bezpieczeństwo (SDN security, firewall, load balancing, fault tolerance) oraz autonomic/adaptive management.

(Shared responsibility) klient vs dostawca

Im wyżej w modelu, tym mniej kontroli klienta:

Warstwa	SaaS	PaaS	IaaS
Aplikacja	dostawca	klient	klient
System op.	dostawca	dostawca	klient
Wirtualizacja	dostawca	dostawca	dost./klient
Sprzęt fiz.	dostawca	dostawca	dostawca

SaaS – minimalna kontrola klienta; **IaaS** – największa odpowiedzialność klienta.

5.3 Harmonogramowanie

(Middleware i warstwa pośrednicząca) IaaS software

Middleware = warstwa oprogramowania między zasobami fiz. a aplikacjami; sieć CPD (klastry węzłów/nodes). **Core middleware**: zarządza infrastrukturą fiz. (hiperwizory, alokacja, load balancing na poziomie fiz.); **user-level middleware**: między runtime a aplikacją.

Platformy zarządzania chmurą (CMP, warstwa IaaS, hiperwizory): VMware (lider wirt.), **OpenStack** (największy otwarty projekt budowy chmur), OpenNebula (open-source, projekty europejskie). **Middleware/orkiestracja**: Kubernetes + KubeFed, koordynator *Aneka* (akademicki). Ekosystemy hybrydowe: Anthos, Azure Arc, AWS Outposts. Uzupełniają: SDN, szyfrowanie, **IaC** (Terraform).

(Zadania i workflow) typy workloadów

Task (zadanie) – podstawowa jednostka. Dwa typy:

- **Batch tasks** (skończone): mają początek/koniec, obliczenia (sekundy), mało wrażliwe na wahania → algorytmy **harmonogramowania**.
- **Web services** (ciągłe): krótkie żądania wrażliwe na opóźnienia (ms) → **load balancing**.

Batch wg: **zależności** – *embarrassingly parallel* (niezależne) vs **work-flow** (DAG – *directed acyclic graph*; węzły=zadania, krawędzie=zależności danych); liczba workflow (jeden vs wiele); heterogeniczność. **Scientific workflows**: HPC, nauka (LIGO, Montage, CyberShake).

Notacja **Grahama** (3-polowa): typ zadań | typ zasobów | kryterium (funkcja celu).

(Model matematyczny) makespan

$$\begin{aligned} \min C_{max} \\ \sum_i x_{ij} &= 1 \quad \forall j \quad (1 \text{ zad.} \rightarrow 1 \text{ maszyna}) \\ \sum_j r_{jk} x_{ij} &\leq C_{ik} \quad \forall i, k \quad (\text{pojemność}) \\ \sum_j t_{ij} x_{ij} &\leq C_{max} \quad \forall i \quad (\text{makespan}) \end{aligned}$$

n zadań j (niezależne, niepodzielne) na m maszyn i . r_{jk} – zapotrzebowanie zad. j na zasób typu k ; C_{ik} – pojemność; t_{ij} – czas wykonania; $x_{ij} \in \{0, 1\}$. Cel: min **makespan** (całk. czas do końca ostatniego zadania). Rozszerzenia: priorytety (wagi), koszty (*facility location*), zależności (*job shop*), dynamika. Rozwiązanie: solwery całkowitoliczbowe → kosztowne ⇒ heurystyki.

(Heurystyki harmonogramowania) Min-Min, SPT, LPT

SPT / SJF (Shortest Processing Time / Shortest Job First): najkrótsze zadania pierwsze; minimalizuje śr. czas oczekiwania. Wada: *głodzenie* długich zadań.

LPT / LJF (Longest Processing Time): najdłuższe zadania pierwsze; szybkie zwolnienie zasobów, unikanie blokady systemu. Gwarancja dla makespan (jednorodne maszyny): $\leq (4/3 - 1/m) \cdot OPT$.

LLMF (Least Loaded Machine First): zadanie → najmniej obciążona VM (najniższe użycie CPU/RAM).

First Fit (FF): pierwsza VM z wystarcz. zasobami (ryzyko fragmentacji).

Best Fit (BF): VM z najmniejszą resztą po przydziale (mniej fragmentacji, więcej przeglądania).

Min-Min – rozszerzenie SPT na heterogeniczne maszyny (t_{ij} znane):

1. *Pierwsze Min*: dla każdego nieprzydzielonego j policz najwcześniejszy czas zakończenia ECT_j (Earliest Completion Time).
2. *Drugie Min*: wybierz zadanie o **minimalnym ECT** (minECT), przypisz do jego najefektywniejszej maszyny, zaktualizuj ECT pozostałych.
3. Powtarzaj aż wszystkie przydzielone.

Faworyzuje krótkie zadania (jak SPT ⇒ może opóźniać długie). Prosta wersja ignoruje koszty VM.

(Kryteria optymalizacji + Green Cloud) funkcja celu

- **Koszt finansowy** (zależny od modelu płatności).
- **Makespan** – najpowszechniejsza metryka wydajności.
- **Koszt obliczeniowy** (CPU, RAM, GPU, przepustowość).
- **Sprawiedliwość** (fairness) – maksymalizacja najgorszego osiągnięcia.
- **Zużycie energii** → **Green Cloud Computing**: minimalizacja energii, przesuwanie energochłonnych zadań na *doliny cenowe* / okresy nadprodukcji OZE (fotowoltaika, w środku dnia/nocy).
- **Wykorzystanie zasobów** (redukcja czasu bezczynności VM = koszt).
- **Niezawodność / dostępność** (skuteczne ukończenie zadania). Bezpieczeństwo, elastyczność – uzupełniające.

(Typy planowania) **Online** (na bieżąco: OLB, MET, MCT) vs **offline/wsadowe** (zbiorczo: Min-Min, Max-Min). **Z wywłaszczaniem** (przerwanie+migracja) vs **bez**. **Scentralizowane** vs **rozproszone**. **Aprowizacja**: on-demand / rezerwacja / spot. Auto-narzędzia: AWS Auto Scaling, Azure Autoscale, Google Autoscaler, Kubernetes (Virtual Kubelet / Cluster Autoscaler).

5.4 Wyzwania i ograniczenia

(Ograniczenia chmury) challenges

- **Vendor lock-in:** różni dostawcy = różne API, architektury, bazy (Google Bigtable, Amazon Dynamo) ⇒ brak wspólnego interfejsu, trudna migracja danych/usług. Łagodzenie: multi-cloud, standardy.
- **Lokalizacja danych:** użytkownik nie wie, gdzie są dane ⇒ trudna kontrola integralności, trudne ACID przy rozproszeniu geo.
- **Niepewność wydajności:** niestabilna przy szczytach; internet wolniejszy niż światłowód/Ethernet. Typy niepewności: moc obliczeniowa, przepustowość, czas dostępu do zasobu.
- **Bezpieczeństwo i prywatność:** dostęp adm. przez internet; geo-rozproszenie → różne jurysdykcje (ryzyko niezgodności). **Segregacja danych:** VM różnych klientów na 1 serwerze → wyciek między VM. Brak gwarancji usunięcia danych.
- **Dostępność:** przerwa w usłudze / awaria internetu / przeglądarki = brak dostępu.
- **Brak standardu SLA** i ogólnych standardów interoperacyjności (prace: Open Grid Forum, Open Cloud Consortium).
- **Limity przydziału:** dostawca ogranicza liczbę jednoczesnych zadań/instancji (limity zasobów, polityki fairness, SLA).

(Fundamenty IoT) Internet of Things

IoT = podłączenie do internetu (sieci przewodowe + bezprzewodowe) wielkiej liczby urządzeń generujących dane. Zbiór danych w **czasie rzeczywistym** → zautomatyzowane systemy → bieżąca kontrola i błyskawiczne reagowanie.

Ewolucja (fale): (1) sprzęt – z RFID; (2) protokoły komunikacyjne; (3) zbieranie danych, **przetwarzanie**, bezpieczeństwo.

Po co przetwarzanie danych IoT? Aplikacje IoT (e-zdrowie, smart grid, e-mobility, rolnictwo) mają **ścisłe wymagania QoS** i są wrażliwe na opóźnienia, a wymagają więcej zasobów niż same urządzenia IoT. Oparcie wyłącznie na chmurze (scentralizowanej, odległej geograficznie, kosztownej) obniża efektywność i budzi obawy o prywatność.

Urządzenia jako elementy architektury Fog/Edge: czujniki, siłowniki, telefony komórkowe (jednocześnie IoT i węzeł Fog!), tablety, inteligentne liczniki/pojazdy/kamery, laptopy, komputery stacjonarne, Raspberry Pi, routery, przełączniki, bramy IoT, serwery high-end, dekodery. Węzeł Fog = dowolne urządzenie, które **przechowuje, przetwarza dane i łączy się z siecią**.

6.1 Topologie: Fog vs Edge

(Kontinuum Cloud-Fog-Edge) 4 warstwy

Wielowarstwowe kontinuum (warstwy NIE są obowiązkowe – konfiguracja zależy od scenariusza, np. sam Fog bez chmury, lub kilka poziomów Edge):

1. **Cloud:** fizyczne węzły DC + VM; charakter: CPU, pamięć, przepustowość; **teoretycznie nieskończone zasoby**, wirtualizacja, elastyczne dopasowanie.
2. **Fog (mgła):** między Edge a chmurą; komponenty fizyczne (bramy, przełączniki, routery, serwery) i wirtualne (cloudlety, kontenery); zarządzanie danymi, usługi komunikacyjne; SDN/NFV.
3. **Edge (brzeg):** bramki między IoT a chmurą/mgłą; telefony, tablety, liczniki, pojazdy, PC.
4. **IoT:** urządzenia końcowe (czujniki, siłowniki) – łączność bezprzewodowa.

(Fog vs Edge – kluczowa różnica) gdzie leży zarządzanie

Edge (starszy koncept): zarządzanie + przetwarzanie **bezpośrednio w/przy** urządzeniu końcowym (sensor, sterownik PAC, kamera). Struktura **płaska** – urządzenia jako niezależne mikrosystemy. Urządzenia podłączone fizycznie do PAC; PAC decyduje: lokalnie czy do chmury. Mniej punktów awarii, oszczędność czasu, mniejsza złożoność – ale **kosztem skalowalności**.

Fog: przesuwania zarządzanie na poziom **sieci lokalnej** – architektura **hierarchiczna, wielowarstwowa** (węzły Fog / bramy IoT). Oprócz obliczeń i przesyłu zajmuje się **przechowywaniem, sterowaniem, przyspieszaniem**. Wiele łączy = **wiele punktów awarii**.

Kroki komunikacji w Fog: (1) do punktów we/wy PAC (program sterujący) → (2) **brama protokołu** konwertuje dane do formatu std. (np. HTTP) → (3) **węzeł Fog** w LAN: analiza + ewent. transmisja do chmury (trwałe przechowywanie).

6.2 Kompromisy inżynierskie

(6 cech Fog/Edge (NSC) + korzyści/problemy)

6 cech (Raport NSC):

1. **Wiedza o lokalizacji + małe opóźnienia** (węzły blisko urządzeń).
2. **Rozproszenie geograficzne** (proxy wzdłuż dróg/torów); trudniejsza migracja VM/kontenerów niż w chmurze.
3. **Heterogeniczność danych i łączności**.
4. **Interoperacyjność i federacja** (współpraca dostawców).
5. **Interakcje w czasie rzeczywistym** (nie wsadowe).
6. **Skalowalność i elastyczność (zwinność) klastrów** – ale **mocno ograniczone zasoby HW** → ograniczamy liczbę VM/kontenerów; lekkie techniki: LXC, Docker, unikernels.

Korzyści: redukcja opóźnień/latencji, oszczędność pasma sieciowego, niższe zużycie energii, lepsza prywatność, wsparcie mobilności.

Problemy: **Mobilność** (protokół LISP – oddziela tożsamość hosta od lokalizacji; częsta migracja VM; mobilne węzły Edge: drony, pojazdy). **Niezawodność i bezpieczeństwo** – węzły brzegowe zawodne, podatne na awarie/partycje sieci i ataki (obsługa przez osoby trzecie/mniej kompetentne).

Wymagane atrybuty węzłów: Autonomia, Heterogeniczność, Hierarchiczne grupowanie, Zarządzalność (orkiestracja), Programowalność.

6.3 Kluczowe technologie i oprogramowanie

(Orkiestracja i rozwiązania chmurowo-brzegowe)

KubeEdge: otwartoźródłowa platforma rozszerzająca Kubernetes na brzeg. **Control Plane w chmurze**, na urządzeniach IoT lekkie **agenty wykonawcze**. Wbudowana obsługa protokołów IoT (MQTT), **praca offline** (gdy węzeł traci łączność).

K3s: odchudzona dystrybucja Kubernetes – cały system zarządzania kontenerami działa **lokalnie**, nawet na pojedynczym Raspberry Pi. **Pełna niezależność lokalnego klastra**.

Oba radzą sobie z niestabilną łącznością, bezpieczeństwem na brzegu i specyfiką zasobów; uruchamianie logiki biznesowej / AI na krawędzi → minimalizacja opóźnień i pasma.

OpenStack (z Bloku 4): platforma chmury prywatnej/hybrydowej typu IaaS; modułowa → zarządzanie zasobami w dużych instalacjach Fog/Edge.

StarlingX: wyspecjalizowany, odchudzony projekt OpenStack. Architektura **Distributed Compute Node (DCN)**; komponenty Nova (wirtualizacja), Neutron (sieć) adaptowane do rozproszonych węzłów brzegowych. Tworzy lokalne „**mikrochmury**” – autonomiczne na terenie fabryki, integrujące się z centralnym DC.

OpenEdge (Progress): warstwa aplikacyjna (ERP/MES, transakcyjne bazy danych), praca offline – nie jest orkiestratorem kontenerów.

6.4 Szeregowanie zadań w Fog/Edge

(Planowanie przy ograniczonych zasobach) scheduling

Zasoby w Edge / zewn. krawędzi Fog są **silnie ograniczone i heterogeniczne** (różne HW, protokoły, architektury). Znaczenie ma **ograniczenie przesyłu** między węzłami (pomijane w chmurze).

Kluczowe wyzwania:

- **Offloading** – decyzja: usługa w chmurze vs warstwa Fog (bramy decydują wg priorytetu/terminu; zadania wrażliwe na opóźnienia → Fog, tolerujące → chmura).
- **Mobilność węzłów** (smartfony, drony, pojazdy) → migracje VM/kontenerów.
- **Heterogeniczność zasobów** (różne MIPS, pobór mocy, łącza).

Kryteria (wielokryterialne, przeciwstawne): efektywność energetyczna, QoS, koszt, czas (opóźnienie), dotrzymanie **deadline**.

Algorytmy: statyczne/oparte na priorytetach – FIFO/FCFS, SPT/SJF, EDF, Round Robin; ulepszone GfE (Greedy for Energy – najtańszy energetycznie węzeł); Min-Max, Min-Min, MCT; metaheurystyki (genetyczne, ML) – dobre wyniki, ale wolne (problem przy sztywnych deadline).

6.5 Model matematyczny i algorytm PSG

(Opóźnienie zadania (delay)) transmisja+obliczenia

$$delay_{ij}[s] = \frac{Size_i[MB] \cdot 8}{Bandwidth_j[Mbps]} + \frac{Length_i[MI]}{ProcRate_j[MIPS]}$$

Czas **przesyłu** (B→b: x8) + czas **obliczeń**. $Size_i = Size_i^{in} + Size_i^{out}$ (tryb batchowy).

(Czas reakcji R_{ki} (zadanie k na węzle i)) 4 składowe

$$d_k^{prop} = \sum_{i,j} 2 e_{ij}^p y_{ij}^k \quad (\text{propagacja})$$

$$d_k^{trans} = \sum_{i,j} \frac{\tau_k^{size} [KB] \cdot 8}{e_{ij}^b [Mbps]} y_{ij}^k \quad (\text{transmisja})$$

$$d_k^{exe} = \sum_i \frac{\tau_k^{length} [M] \cdot 1000}{F_i^{CPU} [MIPS]} x_{ki} \quad (\text{wykonanie})$$

$$d_k^{wait} = \sum_i \sum_l d_i^{exe} z_{kl} x_{ki} x_{li} \quad (\text{kolejka})$$

Zadanie $\tau_k = \langle \tau_k^{size}, \tau_k^{deadline}, \tau_k^{length} \rangle$. Węzeł $F_i = \langle F_i^{CPU}, F_i^{act}, F_i^{idle} \rangle$. Łączy i -j: e_{ij}^p (ms), e_{ij}^b (Mbps). Zmienne: x_{ki} (przypisanie), y_{ij}^k (routing), z_{kl} (priorytet l nad k), s_k (przekroczony deadline). $R_{ki} = d_k^{prop} + d_k^{trans} + d_k^{exe} + d_k^{wait}$.

(Energia, cel, ograniczenia (MINLP)) minimalizacja energii

$$A_i = \sum_k d_k^{exe} x_{ki}, \quad M = \max_i \{A_i\}, \quad t_i = M - A_i$$

$$\varepsilon_i[J] = 0.001 \cdot A_i F_i^{act} + 0.001 \cdot t_i F_i^{idle}$$

$$E^{Total} = \sum_i \varepsilon_i \rightarrow \min$$

$$\text{s.t. } \sum_i x_{ki} = 1; \quad d_k^{prop} + d_k^{trans} + d_k^{exe} + d_k^{wait} \leq \tau_k^{deadline}$$

A_i = czas pracy węzła (suma wykonan); t_i = bezczynność (makespan M minus praca); 0.001: ms $\rightarrow$ s. Cel: **min zużycia energii** (aktywność + bezczynność) przy: każde zadanie na 1 węzle + **dotrzymanie terminów**.

(Algorytm PSG (Priority-aware Semi-Greedy)) Pół-zachłanny (do reguły zachłannej dodano losowość). Uruchamiany przez Fog Controller (FC) co ustaloną liczbę ms. Kroki dla zadania (wg priorytetu = bliskość terminu):

- Sortuj zadania wg **deadline** (najwyższy priorytet = najbliższy termin).
- Dla zadania wyznacz R_{ki} dla każdego węzła FN; **odfiltruj** węzły niespełniające deadline.
- Z pozostałych C węzłów utwórz **krótką listę kandydatów** o najmniejszym ε_i (rozmiar $1..C$ wg strategii randomizacji).
- Przypisz zadanie do węzła z listy **losowo**.

PSG-M = ulepszona wersja. Porównanie z FCFS/EDF/GfE/Detour: **wysoka oszczędność energii** przy dotrzymaniu terminów, podobny makespan. Strategia „zero” randomizacji $\rightarrow$ zawsze min energii.

(Przykład liczbowy (intuicja)) 8 zadań, 3 węzły FN (każdy z każdym, łączy 1000 Mbps $\rightarrow$ przesył nieistotny). Iteracyjnie wybiera się zadanie o najbliższym deadline, liczy R na każdym węzle, odrzuca spóźnione, wybiera **najtańszy energetycznie**. Wynik: zadania omijają szybki, ale energochłonny N1 (gdyby optymalizować sam makespan: 306 ms, ale więcej energii). **Wniosek**: uwzględnienie energii bezczynności daje nieoczywiste, energooszczędne przydziały. Ograniczenie modelu: 1 zadanie/węzeł naraz (znosi je lekka wirtualizacja: KubeEdge, K3s).

(Big Data — pojęcie) problematyka

Wraz z rozwojem internetu Google, Yahoo i in. stanęły przed wyzwaniem przetwarzania **ogromnych zbiorów danych** (tera-/petabajty), pojawiających się z **różną prędkością** (interakcje użytkowników) i w **różnym formacie** (nie zawsze ustrukturyzowane: tekst, obraz). Określenie **Big Data** (pocz. lat 2000) definiowane przez kluczowe cechy.

(Model 3V) rdzeń definicji

- Volume** (Objętość) — ilości danych przekraczające możliwości tradycyjnych narzędzi (relacyjnych baz).
- Velocity** (Prędkość) — szybkość generowania, przesyłania i przetwarzania danych, często w czasie rzeczywistym.
- Variety** (Różnorodność) — różne formaty i źródła: od strukturalnych (bazy) po niestrukturalne (tekst, wideo, IoT).

Rozszerzenia (V5): Veracity (Wiarygodność — jakość/niepewność, dane niekompletne/błędne), **Value** (Wartość).

(Ograniczenia baz relacyjnych) Relacyjne bazy nie sprostały wyzwaniom 3V:

- Skalowalność**: tylko skalowanie **pionowe** (mocniejszy serwer) — szybko granica fizyczna/ekonomiczna.
- Szytywny schemat**: ścisła struktura wymagana przed zapisem.
- Koszty/wydajność**: drastycznie wolniejsze analitycznie (OLAP); mechanizmy spójności **ACID** to „wąskie gardło” przy masowym, równoległym zapisie/odczytzie.

7.1 MapReduce

(MapReduce — co to) Google, 2004

Model przetwarzania równoległego dużych zbiorów danych w klasach (J. Dean, S. Ghemawat). Nazwa od funkcji map i reduce z programowania funkcyjnego; pierwotnie w C++ na Google File System. Najznana implementacja: open-source **Apache Hadoop** (z HDFS).

(Główne cechy)

- Równoległość i skalowalność** — liniowe skrócenie czasu przez dodanie węzłów.
- Odporność na awarie** — master automatycznie przydziela zadanie z uszkodzonego węzła innej maszynie.
- Lokalność danych** — przetwarzanie tam, gdzie dane (np. w HDFS), minimalny ruch sieciowy.
- Prostota** — abstrakcja (map/reduce) ukrywa sieć, synchronizację, przesył.
- Optymalizacja kosztów** — tani, standardowy sprzęt zamiast superkomputerów.

Zastosowania: zliczanie słów, rozproszony grep (reduce=identyczność), licznik odwołań do URL, graf odwrotnych odsyłaczy, odwrotny indeks.

(Fazy przetwarzania (3 fazy)) exam-critical

- Map** — dane wejściowe dzielone na fragmenty; funkcja map tworzy pary **klucz-wartość** (np. dla słowa: (słowo, 1)).
- Shuffle & Sort** (tasowanie/sortowanie) — etap pośredni **zarządzany przez framework**: wyniki map grupowane wg klucza i sortowane; podział na R części przez hash (klucz) mod R i przesył do reduktorów.
- Reduce** — funkcja reduce dostaje klucz i listę wartości (np. ("big", [1,1,1])), agreguje (np. suma) $\rightarrow$ wynik ("big", 3). Programista dostarcza tylko map i reduce.

(Architektura (master-worker))

- Klient** — przesyła zadanie (logika map/reduce, ścieżki we/wy).
- Master** — przyjmuje zadanie, dzieli na części (mapujące/redukujące), przypisuje je workerom, nadzoruje postęp, ponawia zadania po awarii.
- Workers** — maszyny wykonujące zadania map/reduce; map uruchamiany na węzle z lokalnym fragmentem danych (lokalność).

(Całościowy proces zadania)

- Inicjalizacja**: klient wskazuje dane, funkcje, liczbę zadań M (map) i R (reduce), miejsce wyjścia.
- Podział**: master dzieli wejście na M części, uruchamia map na M workerach.
- Map**: każdy worker emituje pary (słowo, 1); zapis lokalny.
- Shuffle&Sort**: po zakończeniu map framework dzieli pary na R części (hash mod R), przesyła do R reduktorów; sortuje i grupuje w listy.
- Reduce**: R workerów sumuje listy wartości.
- Zapis**: wyniki w R plikach wyjściowych.

(Funkcja łącząca (combiner) — opcjonalna) Shuffle&Sort = **najkosztowniejsza** faza (przesył gigabajtów). **Combiner** (zwykle = reduce) robi **wstępną agregację lokalnie na mapperze** przed wysłaniem. Np. zamiast $1000 \times ("klucz", 1)$ wysyła jedno $("klucz", 1000)$. Działa tylko gdy operacja **przemienne i łączna**: suma/zliczanie — TAK; **średnia** — NIE (średnia ze średnich $\neq$ średnia całości).

(Odporność na awarie — utrata workera) exam-critical

- Wykrycie**: master odbiera **heartbeats**; brak sygnału = awaria.
- Ponawianie**: master przypisuje nieukończone zadania map/reduce z utraconego węzła innym, działającym workerom.
- Dostępność danych**: wejście w HDFS replikowane na wielu węzłach $\rightarrow$ ponowne przetworzenie możliwe.

Wymóg DETERMINISTYCZNOŚCI: ponieważ zadania są ponawiane, funkcje map i reduce muszą być **deterministyczne** — dla tych samych danych zawsze ten sam wynik.

(Zabezpieczenie mastera) Początkowo master = **pojedynczy punkt awarii (SPOF)** — jego awaria zatrzymywała cały system i wymuszała restart obliczeń (akceptowalne: awaria 1 mastera « awaria 1 z tysięcy workerów). Późniejsze mechanizmy:

- **Checkpointing:** master okresowo zapisuje stan → wznowienie od punktu kontrolnego.
- **Tryb HA:** aktywny + zapasowy (standby) master; standby przejmuje przy awarii.

(Maruderzy (stragglers)) Pojedyncze zadania map/reduce trwające **znacznie dłużej** niż reszta — wąskie gardło (reduce nie startuje, dopóki nie skończą się wszystkie map). **Rozwiązanie: wykonanie spekulacyjne** (speculative execution / backup): framework wykrywa spóźnialskie zadanie i uruchamia jego **dodatkową, równoległą kopię** na wolnym węźle. Liczy się wynik **pierwszej ukończonej** kopii, druga zostaje przerwana.

7.2 HDFS

(Apache Hadoop) co to + komponenty
Open-source framework (Apache Software Foundation) do **niezawodnych, skalowalnych obliczeń rozproszonych**; rozproszone przechowywanie i przetwarzanie Big Data wg modelu MapReduce. Projektowany dla taniego sprzętu; awarie traktowane jako **norma**, obsługiwane automatycznie. Główne komponenty:

- **HDFS** — rozproszony system plików (bloki + repliki).
- **YARN** — menedżer zasobów (CPU, RAM).
- **MapReduce** — model przetwarzania (Map + Reduce).

Inspiracja: publikacje Google (MapReduce, GFS).

(HDFS — główne cechy)

- **Awaria sprzętu = norma** → szybkie wykrywanie błędów i automatyczne odzyskiwanie przez **replikację**.
- **Dostęp strumieniowy** — przetwarzanie wsadowe, nacisk na **wysoką przepustowość**, nie na niską latencję.
- **Duże zbiory** — pliki od GB do PB, skala do setek węzłów.
- **Write-once-read-many** — jednokrotny zapis, wielokrotny odczyt (obecnie też dopisywanie); upraszcza spójność.
- **Przenoszenie obliczeń tańsze niż danych** — liczenie tam, gdzie dane.
- **Przenośność** między heterogenicznymi platformami.

(Architektura HDFS (Master/Slave)) NameNode + DataNode

- NameNode (Master) — **jeden** węzeł; zarządza **przestrzenią nazw** (drzewo katalogów, nazwy, uprawnienia) i metadanymi; zna lokalizację bloków, ale **nie przechowuje danych**. Trzyma pełny obraz w **RAM** (szybki dostęp). Centralny punkt koordynacji.
- DataNode (Slave) — **wiele** instancji; fizycznie przechowują **bloki** danych na lokalnym FS; obsługują odczyt/zapis; tworzą/usuują/replikują bloki na polecenie NameNode; wysyłają **heartbeat i raporty o blokach**.

(Organizacja danych + metadane)

- **Bloki:** pliki dzielone na bloki **128 MB** (wcześniej 64 MB), niezależne jednostki.
- **Rozproszenie** bloków na różnych węzłach → pliki większe niż 1 dysk.
- Każdy blok HDFS = osobny plik w lokalnym FS węzła (ext4, XFS) + metadane bloku (sumy kontrolne, znacznik generacji).

Magazyn metadanych NameNode: fsimage (migawka przestrzeni nazw) + edits (dziennik transakcji). Start: ładuje fsimage do RAM i odtwarza edits. **Checkpoint** okresowo scala fsimage+edits.

(Replikacja danych) exam

Każdy blok replikowany (domyślnie 3×), kopie na różnych węzłach; decyzję podejmuje NameNode z uwzgl. **topologii sieci (rack-aware)**. Rozmieszczenie (r=3):

- kopia 1 — węzeł klienta (lub losowy),
- kopia 2 — inna szafa (rack),
- kopia 3 — inny węzeł w tej samej szafie co kopia 2.

Pipelining: zapis w potoku klient→DN1→DN2→DN3. Zapis zatwierdzony po replikacji na **wszystkich** replikach (strong consistency). **Sumy kontrolne** wykrywają uszkodzenie → odczyt z innej repliki.

(Erasure Coding vs replikacja 3x) ec

$$\text{narzut}_{3\times} = 300\% (= 3.0\times)$$

$$\text{narzut}_{RS(6,3)} = \frac{6+3}{6} = 1.5 \times (\approx 1.4-1.5\times)$$

HDFS 2.x: replikacja 3× → 100 MB danych = 300 MB na dysku. **Erasure Coding (HDFS 3.x):** zapis danych + **bloki parzystości** (algorytm Reed-Solomon); ta sama odporność przy narzucie ~1.5×. Z 6 bloków danych + 3 parzystości odzyskasz dane nawet po utracie **3 dowolnych** bloków. Idealne dla danych "chłodnych" (rzadko używanych). Wada: rekonstrukcja kosztowna obliczeniowo.

(Odporność HDFS — utrata DataNode) exam-critical

- **Wykrycie:** DataNode wysyła **heartbeat co 3 s**; brak sygnału przez ~2 min → węzeł uznany za niedostępny.
- **Odzyskiwanie (replikacja awaryjna):** NameNode zleca kopiowanie brakujących bloków z istniejących replik na zdrowe węzły → przywrócenie współczynnika replikacji. System działa bez przestojów.
- Przy **Erasure Coding:** NameNode zleca **rekonstrukcję** utraconych bloków z bloków danych/parzystości na innych DN (Application-Master YARN ponawia zadania z uszkodzonego DN).

(Odporność HDFS — utrata NameNode) exam-critical

NameNode trzyma krytyczne metadane → jego utrata blokuje cały klaster. Ochrona:

- **Trwałość zapisu:** fsimage + edits na dysku; **redundancja** — zapis w wielu lokalizacjach (osobne dyski, zdalny NFS).
- **Secondary NameNode:** **nie** jest zapasowym! Pomocniczy proces **checkpointingu** (scala fsimage+edits), by edits nie rosło bez końca.
- **High Availability (HA, Hadoop 2.x+):** dwa NameNode **aktywny-standby**; standby ma aktualny stan i przejmuje rolę natychmiast.
- **Synchronizacja stanu:** wspólny magazyn dzienników — **JournalNode / Quorum Journal Manager (QJM)** (lub NFS).
- DataNode wysyłają raporty bloków do **obu** NameNode → błyskawiczne przełączenie. Hadoop 3 dopuszcza **wiele** węzłów standby.

7.3 YARN i ekosystem

(YARN — co to) Hadoop 2.x

Yet Another Resource Negotiator — system **zarządzania zasobami klastra**, "system operacyjny" dla danych. **Oddziela zarządzanie zasobami od przetwarzania** → wiele silników obliczeniowych na tej samej infrastrukturze. W Hadoop v1 wszystko robił JobTracker (SPOF, słaba skalowalność); YARN go rozbił.

(Komponenty YARN) współpraca z MR

- ResourceManager (RM) — **centralny**; przydział zasobów (CPU, RAM) wszystkim aplikacjom. Składa się z **Scheduler** (polityka przydziału) i **ApplicationsManager** (uruchamia/monitoruje AM).
- NodeManager (NM) — na każdym węźle; monitoruje procesy i zużycie zasobów w **kontenerach**.
- ApplicationMaster (AM) — **jeden na aplikację** (np. konkretne zadanie MapReduce); negocjuje zasoby z RM, współpracuje z NM, monitoruje zadanie, ponawia po awarii.
- Container — pakiet zasobów (RAM/CPU) na jednym węźle, w którym uruchamiane jest zadanie.

(Współpraca YARN + HDFS + MapReduce) Podział ról: HDFS = warstwa **składowania** (NameNode "mózg", DataNode "magazynier"); YARN = warstwa **obliczeń**. Typowy węzeł roboczy = DataNode + NodeManager; węzeł główny = NameNode + ResourceManager.

Lokalność danych (kluczowy styk): przy starcie zadania YARN (RM) pyta HDFS (NameNode), na których DataNode leżą bloki, i uruchamia kontenery **na tych samych maszynach** → odczyt z lokalnego dysku, mniej ruchu w sieci.

MapReduce w YARN to **jedna z wielu** aplikacji (obok Spark, Storm) — multi-tenant klaster.

(Ekosystem Apache Hadoop — narzędzia) tool → zastosowanie

- **Spark** — obliczenia **in-memory** (RAM); szybszy od MR, zadania iteracyjne / ML / analiza interaktywna.
- **Hive** — **HiveQL (HQL)** $\approx$ SQL; tłumaczy zapytania na MapReduce/Tez. Big Data dla "ludzi od SQL".
- **Pig** — język **Pig Latin** (wysokopoziomowy); przekształcenia danych bez Javy → kompiluje do MR.
- **HBase** — baza **NoSQL** (column-oriented) na HDFS; **losowy dostęp w czasie rzeczywistym** (odczyt/zapis), miliardy wierszy.
- **Sqoop** — transfer danych Hadoop ↔ **RDBMS**/mainframe (import/eksport przez MR).
- **Flume** — gromadzenie/agregacja **danych strumieniowych** (logi, clickstreams, social media).
- **ZooKeeper** — **koordynacja** systemów rozproszonych: wybór lidera, blokady rozproszone, metadane/konfiguracja.
- **Oozie** — **harmonogram / workflow** zadań Hadoop.
- **Ambari** — monitorowanie, aprowizacja i zarządzanie klastrami (interfejs webowy).